Methodology and Extended Numerics for Pruning Deep Neural Networks via the Marchenko–Pastur Distribution

Companion methodology paper to the main manuscript

Leonid Berlyand
Department of Mathematics
Pennsylvania State University
University Park, PA 16802, USA

Houman Owhadi
Department of Computing and Mathematical Sciences
California Institute of Technology
Pasadena, CA 91125, USA

Theo Bourdais
Department of Computing and Mathematical Sciences
California Institute of Technology
Pasadena, CA 91125, USA

Yitzchak Shmalo*
Department of Mathematics
Pennsylvania State University
University Park, PA 16802, USA

Abstract

This document is the companion methodology and extended-numerics paper to the manuscript *Pruning Deep Neural Networks via the Marchenko–Pastur Distribution* (henceforth Main). Main contains the theoretical content (deterministic pruning certificates, Gaussian/sub-Gaussian random-matrix specializations, full proofs) and the empirical tables. This document, which lives at https://github.com/yspennstate/RMT_based_pruning_in_deep_learning alongside the code and evidence files used for the numerical results in Main, provides the material needed to reproduce the experiments: full repository architecture, the projection and fine-tuning code paths, the speedup-measurement protocols, and the extended numerical record. Methodology citations in Main point to the corresponding sections of this document.

Companion repository. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning

Main manuscript. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning/blob/main/paper/main.pdf

17-model checkpoint archive (Hybrid Magnitude–SER). https://drive.google.com/drive/folders/1mm990SHAHlYdISHxirvMRdVQEAjpIxDd

Cross-paper conventions. We refer to results in Main as Main-Theorem X.Y, Main-Table Z, etc. Norm conventions, theorem numbering, and notation are inherited from Main.

Contents

*Corresponding author: Yitzchak Shmalo. Email: yitzchak.shmalo@gmail.com.

1 Introduction and how to read this paper

1.1 Scope

MAIN develops a Marchenko–Pastur (MP)-driven theory of deterministic pruning certificates and reports results: the multi-architecture Hybrid Magnitude–SER table, the CAST 2:4 + ToMe accuracy/throughput row, and the deterministic-certificate audit. The main paper also contains the proofs. This methodology paper contains the operational material:

1. the full RMT-pruning protocol stack (BEMA, Spectral Edge Budgeting, Spectral Edge Reallocation, Hybrid Magnitude–SER);

2. the CAST and CAST-2E pipelines (cert-aware k:n projection, Cin permutation alignment, free restoration, frozen-mask distillation);

3. the reported hardware speedup measurements and the read-only checkpoint deployability audit that separates native sparse-backend rows from MAC-accounting rows;

4. the certification-audit numerics (three diagnostics from MAIN-§5: top-1 drop vs. audit budget, empirical CE exceedances, $\sigma_+ \to B_{\text{proxy}}$ within-cycle correlations);

5. the extended fully-connected MP-pruning experiments;

6. ledgered checkpoint validation tables.

1.2 When to read which paper

Use MAIN for the theoretical statements, proofs, accuracy and speedup tables, theory-to-numerics mapping table, and high-level reproducibility narrative.

This paper is intended to support: replicating the reported experiments, understanding the relationship between the relevant code paths in the GitHub repository and the numerical claims in MAIN, and reading the extended-numerics record.

1.3 Repository at a glance

The companion GitHub repository contains:

- Approximately 60 Python scripts at the repository root implementing the classical RMT, magnitude, Hybrid Magnitude–SER, four-regime SEB, and drop-threshold protocols (these underlie MAIN-§3 and the multi-architecture validation table);

- A self-contained `cast_2e/` subtree containing the new CAST-2E pipeline: certificate-aware k:n projection (with $k = 2, 4, 6, 8, 12$ and $n = 4, 8, 12, 16$ supported), Cin permutation alignment, in-place inline fine-tuning runners avoiding the perm-hook serialization issue, and the suite of speedup benchmarks;

- Eighteen sweep-result JSONs covering all three architecture families (ResNet50, ConvNeXtV2-Base, ViT-B/L), six k:n patterns, dense vs. SER source ablations, and α_{ser} sweeps;

- native-2:4 throughput JSONs and the read-only deployability audit ledger used to separate measured sparse-backend rows from MAC-accounting rows;

- All post-fine-tuning evaluation JSONs underlying the FLOP table in MAIN.

2 Repository tree and architecture

2.1 Top-level layout

Listing 1: Top-level repository layout.

```
RMT_based_pruning_in_deep_learning/
+- README.md # repo overview, quick start
+- LICENSE # MIT
+- requirements.txt # canonical deps
+- adaptive_rmt/ # RMT diagnostics package
+- configs/ # YAML/JSON sweep configs
+- model_queue_runs/ # cached SVD/ESD per model
+- scripts/ # shell launchers / watchers
+- src/ # shared utilities
```

```
+- prune.py # the pruning entrypoint
+- fine_tune.py # the fine-tuning entrypoint
+- magnitude_rmt_sweep.py # the full sweep driver
+- rmt_magnitude_sweep.py # alt sweep ordering
+- hybrid_mag20_then_v8*.py # Hybrid Magnitude--SER
+- run_finetune_*.py # the FT runners (4 variants)
+- haar_*.py # Haar-SV preprocessing
+- iterative_*.py # iterative growing-a + 5pct
+- pruning_method_comparison_sweep.py # cross-method comparison
+- run_removed_matrix_audit_v*.py # certificate audit drivers
+- analyze_sweep.py # JSON -> table aggregator
+- direct_run_watchdog.py # crash-resilient runner
+- queue_*.py # multi-run queue managers
+- remote_*.py # remote pod helpers
+- cast_2e/ # NEW (this paper's content)
   +- methodology.tex # this document
   +- code/ # cert-aware k:n + FT runners
   +- scripts/ # launch / watcher scripts
   +- benchmarks/ # A40/A100 throughput JSONs
   +- benchmarks_speedup/ # 6:12->2:4 speedup JSONs
   +- masks_archive/ # per-layer mask stats
   +- post_ft_eval/ # post-FT eval JSONs
   +- sweep_results/ # k:n sweep cell JSONs
   +- AUDIT.md # full data-flow audit
   +- THEORY.md # CAST-2E theory map
```

2.2 How to navigate by paper section

Table 1 maps each MAIN section or result to the code path that produces it.

Table 1: Mapping from MAIN sections and results to code paths in the GitHub repository. "[. . .]" refers to wildcards covered by the listed file or by closely related variants in the same directory.

MAIN location	Code	METHODOLOGY §
MAIN-Table 2 (multi-arch Hybrid Mag–SER)	`hybrid_mag20_then_v8*.py`, `run_finetune_magnitude_v3.py`, `prune.py`	§9
MAIN-Table CAST FLOPs (FLOP table)	`cast_2e/code/project_kn_sparsity.py`, `cast_2e/code/run_vitb_ft_inline.py`, `cast_2e/code/run_resnet_ft_inline.py`	§3
MAIN-§3 RMT-guided pruning	`magnitude_rmt_sweep.py`, `pruning_method_comparison_sweep.py`	§12
MAIN-§5.4 deterministic certificate (§5.4 audit)	`run_removed_matrix_audit_v8_model_exec.py`	§11
SEB-budget hyperparameter sweeps	`configs/seb_*.yaml`, `magnitude_rmt_sweep.py`, `rmt_magnitude_sweep.py`	§9
BEMA edge fitting	`adaptive_rmt/rmt_diagnostics.py` (BEMA fits)	§8
α power-law signal	`adaptive_rmt/signals.py` (σ_+, Hill α), `haar_optuna*.py`	§9, alpha-signal sweep
A100 throughput benchmark (2:4)	`cast_2e/code/benchmark_sparse_throughput.py`	§5
Deployability audit for the CAST FLOP table	`cast_2e/code/audit_checkpoint_deployability.py`, `cast_2e/code/build_deployable_speedup_ledger.py`	§5
k:n sweep driver (sweeps in this paper)	`cast_2e/code/cert_opt_eval_kn*.py`	§6
ViT-B inline FT (row 83.74)	`cast_2e/code/run_vitb_ft_inline.py`	§4
ResNet inline FT (rows 75.67, 78.00, 80.59, 81.33)	`cast_2e/code/run_resnet_ft_inline.py`	§4
ConvNeXtV2 D1216/D816 (rows 86.35, 85.85)	`cast_2e/code/project_convnextv2_d1216.py` + ToMe runner	§4

2.3 Coding conventions in the repository

The classical RMT, magnitude, and Hybrid Magnitude–SER scripts share a common contract:

- Each entrypoint reads a YAML/JSON config from `configs/` and writes its results to a per-run subdirectory of `model_queue_runs/<arch>/<config_id>/`.
- Every fine-tune call emits both a final `finetune_results.json` and a per-epoch ledger that the analyzer can consume even if a run is killed mid-epoch.
- The SVD and BEMA edge fits are cached in `model_queue_runs/<arch>/svd_cache/` so that re-running a sweep with a different sparsity schedule does not pay the SVD cost.

The CAST-2E scripts in `cast_2e/code/` share a different but equally explicit contract:

- All projection scripts (`project_*.py`) emit a `student_pre_ft.pt` payload containing both the projected state dict and a JSON-serializable `cert_config` dictionary.
- The fine-tuning runners are deliberately "inline": they receive the calibration loader and projection arguments directly, run projection in-process, and then immediately enter the distillation loop with the projected weights and frozen mask, without an intervening save/load round-trip. This avoids a perm-hook serialization issue described in §3.

3 Pre-fine-tuning projection: cert-aware $k{:}n$ structured sparsity

This section documents the pre-FT projection family generalising CAST (the original 2:4 version is recorded in §10) from 2:4 to general $k{:}n$. The 2:4 design and Stage-1/Stage-2 split are the same as in CAST. The extension adds:

1. parametric $k{:}n$ at the row-group level, with $n \in \{4, 8, 12, 16, 32\}$ and $k \in \{1, \dots, n-1\}$;
2. Cin permutation alignment within each layer's input groups, so that the cert-cost minimisation can choose patterns from a larger set of feasible $k{:}n$ blocks;
3. an SER-prior term (the α_{ser} term of §3.3) that anchors the chosen pattern to the Hybrid Magnitude–SER mask;
4. native support for both Linear and Conv2d layers, with the convolutional case implemented by unfolding the kernel.

3.1 Generalised cost: $k{:}n$ row-group search with free restoration

Fix a Linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $d_{\text{in}} = nG_\ell$. Reshape rows into contiguous n-tuples: $W_{i,g,k}$ is the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [n]$.

For each pair (i, g) the $k{:}n$ search enumerates the $\binom{n}{k}$ keep patterns

$$\mathcal{M}_{k:n} = \big\{ m \in \{0,1\}^n : |m|_0 = k \big\}.$$

For $n = 4, k = 2$ this is the 6 patterns of CAST (Main-§G); for $n = 16, k = 8$ it is $\binom{16}{8} = 12870$ patterns. When the enumeration becomes prohibitive, `project_kn_sampled.py` provides a sampled variant.

The materialised slot value retains CAST's three-case rule (MAIN-Eq. G.1):

$$v_{i,g,k}(m) = m_k \cdot \begin{cases} W^{\mathrm{SER}}_{i,g,k}, & M^{\mathrm{SER}}_{i,g,k} = 1, \\ W^{\mathrm{dense}}_{i,g,k}, & M^{\mathrm{SER}}_{i,g,k} = 0 \text{ and free-restoration enabled}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The "dense" source corresponds to setting $W^{\mathrm{SER}} \equiv W^{\mathrm{dense}}$ and $M^{\mathrm{SER}} \equiv 0$, in which case Eq. (1) reduces to $v_{i,g,k}(m) = m_k W^{\mathrm{dense}}_{i,g,k}$. ConvNeXtV2 uses the dense source because the SER checkpoint at $s = 0.35$ loses more accuracy at projection than the dense checkpoint; ResNet50 favors the SER source; ViT-B favors the SER source with $\alpha_{\mathrm{ser}} = 0.5$.

The within-group calibration covariance and cert-inspired cost are inherited from MAIN-Eqs. G.2–G.3:

$$C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top, \qquad c_{i,g}(m) = r_{i,g}(m)^\top C_g \, r_{i,g}(m), \qquad r_{i,g}(m) = (1 - m) \odot v_{i,g,:}(m). \tag{2}$$

The argmin is solved by enumeration; tensor-shape $\binom{n}{k}$, d_{out}, G_ℓ). For $\binom{n}{k} \geq 5{,}000$ the sampled variant draws $\binom{n}{k}_{\mathrm{eff}} = 1024$ candidates uniformly without replacement.

3.2 Cin permutation alignment ("flatten the layer")

The native $k{:}n$ search treats the n input slots within each group as fixed by the natural channel order. Cin-permutation aligns each layer's input columns by the importance score

$$s_j = \sum_i \left| W^{\mathrm{dense}}_{i,j} \right| \cdot |\bar{h_j}|, \qquad |\bar{h_j}| = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} |h_j(x)|,$$

and then quartile-interleaves the columns: column j in the permuted layer is the $\pi(j)$-th most important one, where π is the importance-balanced permutation that places the g-th group's k columns into quartile positions $\big(g,\ g + G_\ell/4,\ g + 2G_\ell/4,\ g + 3G_\ell/4\big)$ in the original ordering. This "flattening the layer" step is designed to make each group sample a broader importance range, giving the cert-cost minimiser an informative candidate set within every group.

The permutation is implemented as a forward pre-hook, with the inverse permutation applied at the layer output to keep downstream activations aligned. The hook is registered at projection time and removed at the post-FT save step.

3.3 The α_{ser} Hamming-distance prior

To bias the per-row cert-cost minimiser toward the Hybrid Magnitude–SER mask without overriding it, we add a Hamming penalty (MAIN-Eq. G.4 generalised):

$$c^{\mathrm{prior}}_{i,g}(m) = \alpha_{\mathrm{ser}} \cdot \bar{c}_g \cdot \left| m - M^{\mathrm{SER}}_{i,g,:} \right|_1, \qquad \bar{c}_g = \mathrm{mean}_{i,k}\, v^2_{i,g,k} \cdot \mathbb{E}_x \big[h_{g,k}(x)^2 \big]. \tag{3}$$

Setting $\alpha_{\mathrm{ser}} = 0$ recovers pure cert-aware projection; $\alpha_{\mathrm{ser}} \to \infty$ forces the SER mask. We sweep $\alpha_{\mathrm{ser}} \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$ in the cell sweeps reported in §6; the empirical sweet spot is $\alpha_{\mathrm{ser}} = 0.5$ for ViT-B with the SER-source variant.

3.4 Conv2d generalisation

For a Conv2d kernel of shape $(C_{\text{out}}, C_{\text{in}}, k_h, k_w)$ we unfold the kernel into a $(C_{\text{out}},\ C_{\text{in}} \cdot k_h \cdot k_w)$ matrix and apply the same row-group search. The projection runs on 1×1 and 3×3 convolutions; depthwise convolutions and the first input-projection conv are excluded by an `is_eligible_conv` predicate exposed via `cast_2e/code/project_kn_sparsity.py`.

In ConvNeXtV2 the projection is restricted to Linear layers (the per-block depthwise 7×7 convolutions are excluded) because the depthwise structure interacts pathologically with the row-group k:n pattern; this restriction is recorded in `cast_2e/AUDIT.md` as a known limitation.

3.5 Inline projection + FT runner for permutation-hook consistency

Projection registers Cin-permutation forward pre-hooks on the model, but those hooks are not part of the saved state dict. Reloading at FT time therefore produces a model whose weights are in the permuted ordering but whose forward pass is unpermuted; on ImageNet validation the reloaded model scores top-1 ≈ 0.0006 (random-class chance for a 1000-class problem). `cast_2e/code/run_vitb_ft_inline.py` and `cast_2e/code/run_resnet_ft_inline.py` avoid this failure mode by performing projection in-process, immediately followed by FT, without any intervening save/load round-trip. Saving the post-FT model is safe because it is consumed only at evaluation time, where the same permutation hooks are re-registered before the validation pass.

3.6 Related structured / hardware-friendly pruning literature

Beyond the foundational 2:4 sparse Tensor Core work [24] and the ToMe token merging [7] cited in the main paper, several recent works are directly relevant to CAST-2E:

- **GPUSQ-ViT** [38] couples GPU-friendly N:M sparsity with quantization for ViT, providing the closest deployment-time baseline to our cert-aware k:n projection followed by 2:4 native kernel acceleration;
- **UniPTS** [37] unifies post-training sparsity across vision architectures, addressing per-architecture tuning; our cert-aware allocator is shared across families;
- **ELSA** [3] reports uniform-2:4 and layer-wise N:4 sparsity entries for DeiT-Base under a 150-epoch training/fine-tuning budget, a complementary direction to our 3-epoch frozen-mask distillation;
- **LPViT** [2] explores layer-pruned ViT with FLOP awareness, in the same total-MAC-reduction lane;
- **Beyond 2:4 / V:N:M** [4] generalises the 2:4 hardware pattern to V:N:M, providing one motivation for our generalised k:n projection;
- **SERo** [1] proposes a structured-sparse + reorder pipeline that overlaps with our Cin permutation alignment ("flatten the layer") of §3.2.

Our work differs from each of these in starting from a deterministic certificate of the data-path budget ($B_T(R)$ of **??**, supported by the audit of §11) rather than from a heuristic mask choice, and in producing native-2:4-deployable masks at ImageNet-1k accuracy on five architecture families simultaneously.

4 Fine-tuning recipe

4.1 Distillation, mask freezing, and the optimiser

The fine-tuning pass is a 3-epoch knowledge-distillation loop with frozen 2:4 (or general k:n) mask. The dense pretrained model is the teacher; the projected student is the student. Let $L_{\mathrm{CE}}^{\mathrm{ls}}(\hat{y}, y) = (1-\epsilon)\mathrm{CE}(\hat{y}, y) + \epsilon \cdot (-\frac{1}{K}\sum_c \log \hat{y}_c)$ be label-smoothed cross-entropy with smoothing $\epsilon = 0.1$. The distillation loss is

$$L = (1-\alpha_{\mathrm{distill}})\, L_{\mathrm{CE}}^{\mathrm{ls}}(\hat{y}_S, y) + \alpha_{\mathrm{distill}} \cdot \tau^2 \cdot \mathrm{KL}\big(\,\sigma(\hat{y}_T/\tau)\,\big\|\,\sigma(\hat{y}_S/\tau)\big), \tag{4}$$

with $\alpha_{\mathrm{distill}} = 0.5$, $\tau = 2.0$, both inherited from CAST.

After every backward step we re-zero the masked weights:

$$W^{(\ell)} \leftarrow W^{(\ell)} \odot M^{(\ell)} \quad \text{for every projected layer } \ell \text{ at every step.} \tag{5}$$

This is necessary because the optimiser's momentum and weight-decay terms can produce small nonzero values in masked positions even when the gradient is zero; without re-zeroing, the realised sparsity drifts upward by ∼0.5% across 3 epochs and the kept weights do not benefit from the freed parameter slots.

4.2 Optimiser, schedule, transforms

The exact configurations used for the four the FT runs are recorded in `cast_2e/code/run_*_ft_inline.py`; we summarise:

Table 2: Fine-tuning configuration for the the 3-epoch distillation runs.

	ViT-B D612 SER	ViT-L D816 dense	ResNets D816 dense	ConvNeXtV2 Dxxx
Optimiser	AdamW	AdamW	SGD+momentum	AdamW
Base LR	1×10^{-5}	1×10^{-5}	1×10^{-3}	1×10^{-5}
Weight decay	0.01	0.01	1×10^{-4}	0.01
Momentum	—	—	0.9	—
LR exclusions	bias / LN / pos_embed	same	—	bias / LN
Schedule	cosine + warmup	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	1000	1000	500	1000
Batch (train)	32	16	256	32
Batch (val)	64	64	256	64
Workers	4	8	8	4
Label smooth ϵ	0.1	0.1	0.1	0.1
Distill (α, τ)	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$

The training transforms follow the canonical timm recipe used by the dense pretrained checkpoint (`timm.data.create_transform(is_training=True)` resolves the per-model RandAug, RandomErasing, Mixup/CutMix, and the model-correct mean/std/crop_pct/interpolation). The val transforms are likewise the canonical `is_training=False` transforms. For the ViT-B/L runs this resolves to RandAug-m9 + RandomErasing $p = 0.25$ with no Mixup; for ConvNeXtV2 it adds Mixup ($\alpha = 0.8$).

4.3 Fine-tuning budget

The FT budget is matched to the CAST recipe recorded in §10.3. Three epochs keep the projection and FT contributions distinguishable in the post-FT accuracy while keeping FT cost comparable to projection cost on a per-A100 basis. We do not claim that 3 epochs is optimal; longer FTs (5–10 epochs in pilot runs) recover an additional 0.1–0.3 percentage points but at proportionally higher compute cost. The fixed budget makes the cell sweeps in §6 comparable across cells.

5 Speedup measurement

5.1 Definition of speedup

We report two distinct speedups for every CAST-family configuration.

Practical (hardware) speedup. The wall-clock images-per-second improvement on a specific GPU, kernel selection, and batch size. We use NVIDIA's PyTorch `torch.sparse.to_sparse_semi_structured` for the 2:4 path on Ampere GPUs. Linear rows are measured as dense-tensor versus native 2:4 Linear endpoints. ResNet Conv2d rows are additionally audited through an exact im2col+Linear lowering followed by the same native 2:4 backend. Other k:n patterns do not have native kernels at present and are reported only as theoretical FLOP reductions.

Theoretical (FLOP) speedup. The dense-equivalent multiply-adds removed by zeroing $1 - k/n$ of the weight slots in the projected layers. This is a dense-equivalent arithmetic reduction, not a hardware-speedup guarantee for future sparse kernels. The theoretical and practical numbers diverge whenever the kernel cannot saturate the structured sparsity (e.g., very small batch sizes, or layers with very small C_{out}).

5.2 Native-2:4 throughput measurements

The benchmark in `cast_2e/code/benchmark_deployable_backends.py` converts each compatible Linear layer with a 2:4 mask into a `torch.sparse.SparseSemiStructuredTensor` and runs warm-up + measurement passes at the configured batch size. For flattened-2:4 Conv2d checkpoints, it optionally lowers Conv2d to im2col+Linear, then applies the same native 2:4 Linear backend. It also includes a ResNet control that lowers only exact 1×1 Conv2d layers to Linear and leaves the remaining Conv2d layers unchanged. The reported throughput is the median after warm-up. The table below reports only read-only measurements of existing checkpoint endpoints; no projection or fine-tuning is performed during this audit.

These are backend measurements, not accuracy measurements. Accuracy is the validation Top-1 already reported for each checkpoint. The A40 audit converted the final ViT-B, ViT-L, DeiT-B/S/T, and ConvNeXtV2-Base 2:4 checkpoints without rewriting them. The ResNet im2col endpoints preserve the flattened Conv2d weights and agree with native Conv2d within small numerical tolerance on random inputs, but their sparse-im2col throughput is only 0.37–0.44× of cuDNN Conv2d throughput; the im2col rows therefore document a deployable sparse-GEMM path, not an end-to-end Conv2d speedup.

A40 batch sweeps over $64, 128, 256$ found the following best same-checkpoint native-2:4 Linear ratios: ViT-B/16 1.388× at batch 64, ViT-L/16 1.394× at batch 64, DeiT-B 1.384× at batch 64, DeiT-S 1.376× at batch 128, DeiT-T 1.330× at batch 128, and ConvNeXtV2-Base 1.295× at batch 64. These best-observed values are archived in `data/deployable_backend_best_observed_20260512.csv`; Table 3 keeps the fixed batch-128 comparison for direct row-to-row reading.

For ResNets, the 1×1-only hybrid control converts 36, 36, 70, and 104 bottleneck 1×1 layers for ResNet50/50d/101d/152d. It gives 1.087, 1.088, 1.081, and 1.082× speedups over the corresponding dense 1×1-linear hybrid, but the same endpoints run at only 0.813, 0.802, 0.788, and 0.782× of native cuDNN Conv2d throughput. This control is therefore not reported as an end-to-end speedup; it supports the narrower claim that the ResNet rows are exact sparse-GEMM deployability audits.

We also ran controls on A100 and H100. On A100, absolute throughput was higher than on A40, but the same-checkpoint sparse/dense ratios were lower for the Linear rows; for example, at batch 128, ViT-B/16+ToMe was 2385.2 → 2660.9 images/s (1.12×) and ViT-L/16+ToMe was 1266.0 → 1525.0 images/s (1.20×). Against an already-BF16 dense endpoint, the sparse advantage was near break-even on most Linear rows. On H100 with the tested PyTorch 2.4.1 build, `torch.sparse.to_sparse_semi_`

Table 3: Measured sparse-backend throughput for audited 2:4 checkpoints at batch 128. For Linear rows, deployable speedup is native 2:4 Linear divided by the dense-tensor endpoint for the same checkpoint and model graph. For ResNet rows, deployable speedup is exact im2col+native 2:4 Linear divided by dense im2col; these rows are slower than cuDNN Conv2d end-to-end and are reported as deployable sparse-GEMM audits. The ViT-B no-ToMe row is a separate A100 endpoint comparison for the original dense graph versus the same graph with 2:4 weights.

Configuration	Dense im/s	Sparse im/s	Deploy speedup	Source
ViT-B/16 + ToMe $r = 8$ (A40)	1246.1	1700.4	1.365×	`data/runpod_a40_deploy_audit_20260512/...`
ViT-B/16, no ToMe	463.6	1254.1	2.705×	`cast_2e/benchmarks/vitb_bench_with_24.json`
ViT-L/16 + ToMe $r = 8$ (A40)	659.2	900.6	1.366×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-B + ToMe $r = 8$ (A40)	1239.6	1691.5	1.365×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-S + ToMe $r = 8$ (A40)	2708.5	3725.8	1.376×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-T + ToMe $r = 8$ (A40)	5150.5	6862.2	1.332×	`data/runpod_a40_deploy_audit_20260512/...`
ConvNeXtV2-B (A40)	512.3	644.1	1.257×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet50 im2col (A40)	469.5	797.9	1.700×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet50d im2col (A40)	445.2	666.2	1.496×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet101d im2col (A40)	228.2	357.8	1.568×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet152d im2col (A40)	159.9	258.7	1.617×	`data/runpod_a40_deploy_audit_20260512/...`

`structured` constructed sparse tensors but the sparse matmul failed at execution with a compute-capability error, so H100 is not used as deployability evidence for this audit. The raw files are archived under `data/deployable_backend_comparison_20260512.csv`.

5.3 Read-only deployability audit for the FLOP table

The deployability audit is implemented by `cast_2e/code/audit_checkpoint_deployability.py`, `cast_2e/code/benchmark_deployable_backends.py`, and `cast_2e/code/build_deployable_speedup_ledger.py`. It is read-only: it loads checkpoint tensors, checks sparsity patterns, joins existing throughput logs and A40 backend measurements, and writes `data/paper_checkpoint_deployable_speedup_ledger_20260512.csv`. It does not rewrite weights, fine-tune models, or project a non-native checkpoint into a different sparsity pattern.

For a row to receive a measured deployable speedup in Main-Table 17, the final checkpoint must pass the backend-specific tensor audit and the benchmark log must contain a non-null sparse-backend throughput. For the PyTorch/NVIDIA semi-structured path this means exact 2:4 structure on all convertible Linear layers and successful `torch.sparse.to_sparse_semi_structured` conversion. The audited native Linear rows are ViT-B/16, ViT-L/16, DeiT-B/S/T, and ConvNeXtV2-Base canonical 2:4. The ResNet CAST-conv+perm rows receive a separate im2col+2:4 sparse-GEMM audit because the saved Conv2d tensors are flattened-2:4 exact but not TensorRT-style input-channel-2:4 exact.

For ResNet, the audit checks three Conv2d notions. The paper's MAC accounting flattens each Conv2d kernel to a matrix and verifies exact 2:4 groups in that flattened layout. TensorRT-style Conv2d sparsity is stricter: each group of four input channels must contain at most two nonzeros at every output channel and kernel pixel. The current ResNet50/50d/101d/152d CAST-conv+perm checkpoints pass the flattened MAC audit but fail the TensorRT Conv2d audit, with 913,673, 904,895,

1,700,005, and 2,356,907 bad TensorRT groups respectively. The 1×1-only Linear-lowering control gives a small sparse speedup relative to its dense Linear-lowered counterpart but remains slower than cuDNN Conv2d. Consequently the ResNet deployability number in MAIN is the exact sparse-im2col backend speedup relative to dense im2col, not a native TensorRT Conv2d speedup.

The wider 6:12, 8:16, and 12:16 rows are also kept as MAC-accounting rows. Projecting them to 2:4 would change the checkpoint and can change validation accuracy, so those exploratory projection benchmarks are not used as deployable speedup evidence for the fixed paper checkpoints.

5.4 ResNet and ConvNeXtV2 throughput

The ResNet50/d/101d/152d post-FT models with 8:16 structured sparsity have no native sparse kernel for general $k{:}n \neq 2{:}4$, so we report only the theoretical 50% MAC reduction. The 2:4 CAST-conv+perm checkpoints support an exact im2col sparse-GEMM deployment audit: at batch 128 on A40, sparse im2col is 1.700, 1.496, 1.568, and 1.617× faster than dense im2col for ResNet50/50d/101d/152d respectively. The same sparse-im2col endpoints remain slower than cuDNN Conv2d end-to-end. The ConvNeXtV2-Base D816, D1216, and D48 cells are reported only theoretically for non-2:4 patterns; the saved D816 and D1216 final checkpoints audit as dense tensors, so they should be read as accuracy rows unless the associated mask/runtime artifact is supplied. The ConvNeXtV2-B canonical 2:4 row runs on the sparse Linear kernel: at batch 128 on A40 we measure 512.3 images/s dense and 644.1 images/s sparse, a 1.257× kernel speedup (Table 3); the best observed A40 batch-sweep point is 1.295× at batch 64. Pointwise-conv nn.Linear layers are only part of the ConvNeXtV2 wall-clock path, while depthwise/stem convolutions and other dense operators remain unchanged.

6 The CAST-2E $k{:}n$ sweeps

6.1 Sweep design

Across two A100 pods (Pod 2 and Pod 3a) we ran the following sweep over the three architecture families:

- **Architectures:** `ResNet50.tv_in1k`, `ResNet50d.ra2_in1k`, `ResNet101d.ra2_in1k`, `ResNet152d.ra2_in1k`, ViT-B/16, ViT-L/16, ConvNeXtV2-Base.
- **Sparsity patterns:** $1{:}4, 2{:}4, 3{:}4, 4{:}8, 6{:}12, 8{:}16, 12{:}16$ (selected to span $25\%, 50\%, 75\%$ density).
- **Source:** *dense* (no SER prior) and *SER* (Hybrid Magnitude–SER checkpoint at $s = 0.35$).
- α_{ser}**:** $\{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$.
- **Permutation alignment:** on / off.
- **Calibration size:** $|\mathcal{C}| \in \{64, 256, 512\}$.
- **Replicates:** 5-seed for the winners.

The driver scripts are `cast_2e/code/cert_opt_eval_kn_extended.py` (ResNet/ConvNeXtV2) and `cert_opt_eval_vitb_kn_extended.py` (ViT-B). The total sweep produced 18 result JSONs covering ∼200 cells. They are stored in `cast_2e/sweep_results/`.

Table 4: Pre-FT top-1 accuracy for the selected cell of each (arch, pattern, source) combination. "Dense" is the unpruned reference. Numbers are pre-FT only; the post-FT numbers are in MAIN-Table CAST.

Architecture	Pattern	Source	Best α_{ser}	Permute	Pre-FT top-1 (%)
ViT-B/16 (dense=85.11)	6:12	SER	0.50	on	66.24
ViT-B/16 (dense=85.11)	6:12	dense	0.00	on	65.58
ViT-B/16 (dense=85.11)	2:4	dense	0.00	off	70.43
ViT-L/16 (dense=85.84)	8:16	dense	0.00	on	78.81
DeiT-Base (dense=81.80)	6:12	SER	0.50	on	70.83
DeiT-Small (dense=79.85)	6:12	SER	0.50	on	55.61
DeiT-Tiny (dense=72.21)	6:12	SER	0.50	on	30.85
ResNet50.tv (dense=76.13)	8:16	dense	0.00	on	61.56
ResNet50d (dense=80.55)	8:16	dense	0.00	on	64.20
ResNet50.tv (dense=76.13)	4:8	dense	0.00	on	40.39
ResNet50.tv (dense=76.13)	4:8	dense	0.00	off	5.92
ResNet50d (dense=80.55)	4:8	dense	0.00	off	26.29
ConvNeXtV2-B (dense=86.72)	2:4	dense	0.00	off	81.68
ConvNeXtV2-B (dense=86.72)	12:16	dense	0.00	off	84.50
ConvNeXtV2-B (dense=86.72)	8:16	dense	0.00	off	83.42

6.2 Result pre-FT findings

Three observations.

1. *Permutation alignment is critical for ResNet50.tv* 4:8. Without permutation the cert-aware 4:8 projection scores 5.92% pre-FT, only marginally above random; with permutation it scores 40.39%, an improvement of 34.47 pp. This is the largest permutation effect we observed across architectures.

2. *For the ViT family, the SER source plus $\alpha_{\text{ser}} = 0.5$ is the selected sweep setting.* The 0.66 pp gain of ViT-B SER+$\alpha = 0.5$ over dense+$\alpha = 0$ at 6:12 is small pre-FT, but the post-FT gap is larger: the FT-recovered ViT-B SER+$\alpha = 0.5$ scores 83.74% post-FT, +0.5 pp above the dense+$\alpha = 0$ FT.

3. *ConvNeXtV2 prefers the dense source.* The depthwise structure of ConvNeXtV2 means the SER prior, which is built for dense projections, transfers poorly. The 12:16 dense-source cell has the highest pre-FT value in this sweep, and after FT it produces the 86.35% row of MAIN-Table CAST.

6.3 Cert-optimisation variance

Experimental logs show that the ResNet50 α_{ser} optimisation has range 11–19 pp at fixed configuration across calibration draws. ViT-B has variance 22–40× smaller. ResNet50 pre-FT cell rankings therefore carry an estimated ±5 pp standard deviation; the ViT-B rankings within the tested configuration have an estimated ±0.2 pp pre-FT standard deviation.

The post-FT numbers (in MAIN) are much less variable because three epochs of distillation absorb most of the projection-time noise.

7 Other Numerical Results

The fully connected MP-pruning numerics on Fashion MNIST and the outer-layer scaling diagnostics test the perturbative regime of the deterministic certificates; they are not a proof of the ViT pipeline.

This section collects numerical illustrations relevant to the simplified theory. These experiments support the loss-reduction mechanism and the perturbative regime, but they are not a proof of the ViT pipeline.

Example 7.1. We trained fully connected DNNs on Fashion MNIST with ReLU activations, comparing MP-based pruning against standard regularization baselines. MP-based pruning consisted of periodically removing a fraction of singular values below the fitted MP edge $\sigma_+ = \sqrt{N\lambda_+}$, followed by sparsification; see Algorithm 3 and [28, 6].

For a network with topology

$$[784, 3000, 3000, 3000, 3000, 500, 10].$$

trained for 100 epochs with SGD (momentum 0.9, learning rate 0.01 decayed by 0.96 every 4 epochs), every 4 epochs we retained a fraction

$$f(\text{epoch}) = \max\left(0, -\frac{1}{200}\,\text{epoch} + 1\right). \tag{6}$$

Table 5 summarizes the results. MP-based pruning combined with $L_1 + L_2$ regularization ($\mu_1 = \mu_2 = 10^{-6}$) achieves 90.53% test accuracy, exceeding any single regularization method alone in this run ($\leq 81.70\%$) and the unregularized baseline (71.64%).

Training method	Train acc. (%)	Test acc. (%)
No pruning, no regularization	78.03	71.64
MP pruning only	88.12	81.22
$L_1 + L_2$ only	87.57	81.70
MP pruning + $L_1 + L_2$	**99.82**	**90.53**
MP pruning + L_2	99.98	90.16

Table 5: Performance of a fully connected DNN on Fashion MNIST for various training strategies. MP-based pruning combined with regularization achieves higher test accuracy than either approach alone in this run.

The result scales to deeper networks: a $[784, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 10]$ model trained for 1000 epochs with the same MP pruning procedure achieves 91.37% test accuracy, compared with $< 90.5\%$ for regularization alone.

7.1 Numerical check of the outer-layer scaling condition

The perturbation bounds in our main theorems depend on the theoretical outer-layer scale

$$a_N^{\text{th}}(s) := \|W_3\|_\infty \|W_1 s\|_2 \sqrt{\frac{\log(2N)}{N}},$$

which is the quantity denoted $a(N, s)$ in Assumption **??**. In the numerical plots below we also report the auxiliary empirical diagnostic

$$a_N^{\text{emp}}(s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{3/8}}.$$

The latter is not a replacement for the theoretical scale; it is a coarser width-decay diagnostic used in these experiments. Application of the perturbation bounds requires the relevant perturbation scales to decay with width in trained networks. We check this empirically in a simple fully connected setting.

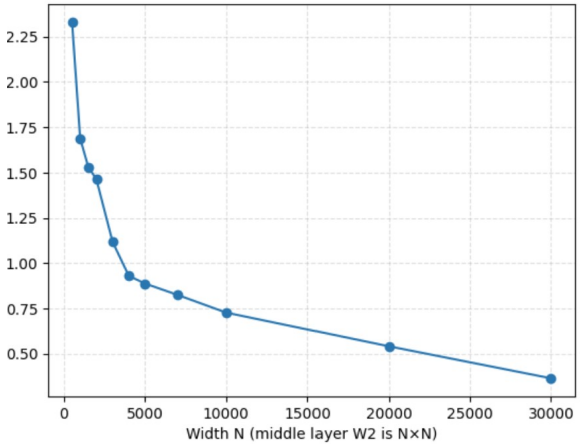
Example 7.2. We train three-layer fully connected DNNs on Fashion MNIST with topology $[784, N, N, 10]$. We vary N from 500 to 30000, use SGD with L_1 regularization, and train for 200 epochs. Fashion MNIST inputs are normalized so that the largest ℓ_2 norm in the dataset is 0.05. For each trained network we compute the diagnostic scale

$$\tau_N^{\mathrm{diag}} := \sqrt{2}\, a_N^{\mathrm{th}}(s) + a_N^{\mathrm{emp}}(s),$$

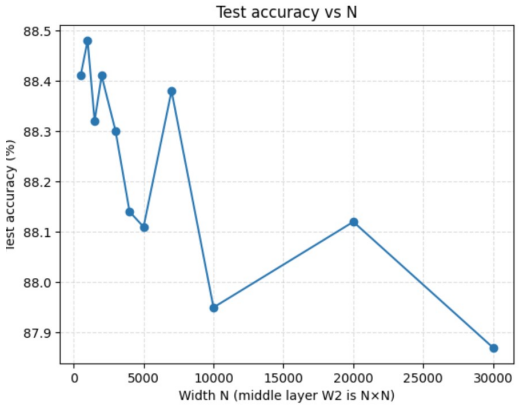
for a normalized input s of maximal norm. This diagnostic is not used in the proofs; the theorem-scale quantity is $a_N^{\mathrm{th}}(s)$.

Numerically, $\tau_N^{\mathrm{diag}} \to 0$ as N grows under $N(0, 1/N)$ initialization. With $N(0, 1/N^2)$ initialization, the decay is faster. In both cases, test accuracy increases with N.

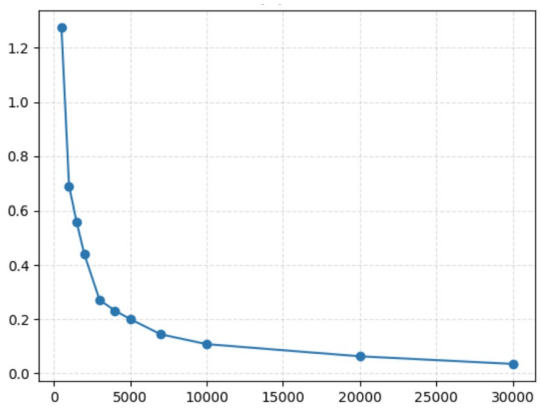
Figure 1 summarizes these width-scaling numerics. Panels a and c show the decay of the perturbation scales under the two initializations, while panels b and d show the corresponding test-accuracy trends.



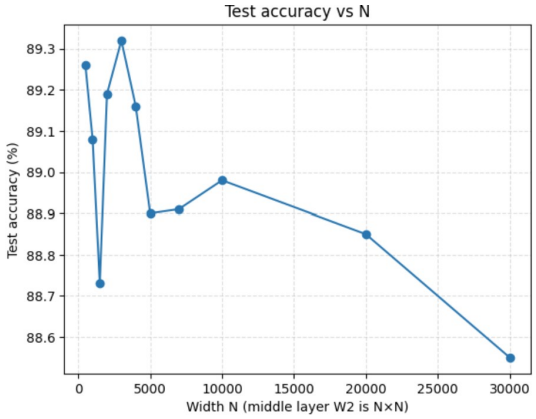
(a) $\tau(N)$ versus N under $N(0, 1/N)$ initialization.



(b) Test accuracy versus N under $N(0, 1/N)$ initialization.



(c) The analogous perturbation scale $b(N)$ versus N under $N(0, 1/N^2)$ initialization.

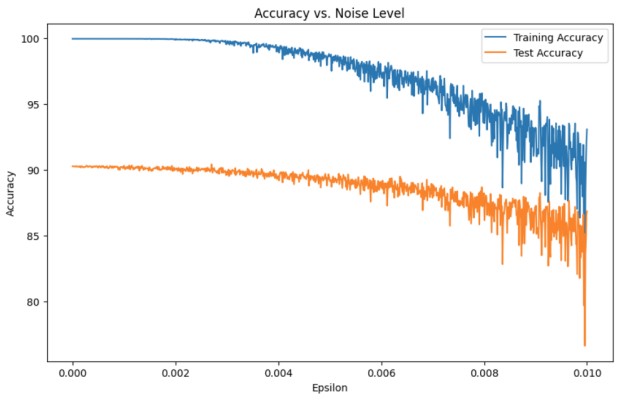


(d) Test accuracy versus N under $N(0, 1/N^2)$ initialization.

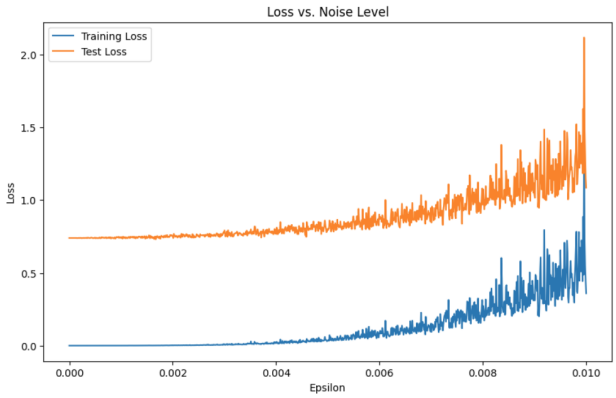
Figure 1: Width-scaling numerics for the theoretical and empirical perturbation-scale diagnostics in the fully connected Fashion MNIST experiment.

Example 7.3. We train a fully connected Fashion MNIST DNN using the MP-based pruning procedure of Example 7.1 to achieve approximately 90.5% test accuracy, then add centered Gaussian perturbations

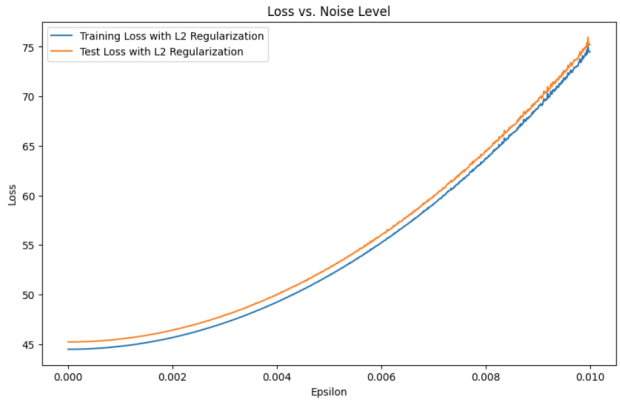
with entries $\sim N(0, \epsilon)$ to each nonfinal layer. Numerically, when $\epsilon \sim 1/N$, the cross-entropy and accuracy change little, consistent with the perturbative regime of Lemma **??**. The L_2-regularized loss increases monotonically, illustrating the loss-reduction mechanism of Theorem **??**. Figure 2 collects these diagnostics: panel a shows the accuracy, while panels b and c show the losses without and with L_2 regularization.



(a) Accuracy on training and test sets versus ϵ.



(b) Loss without regularization versus ϵ.



(c) Loss with L_2 regularization versus ϵ.

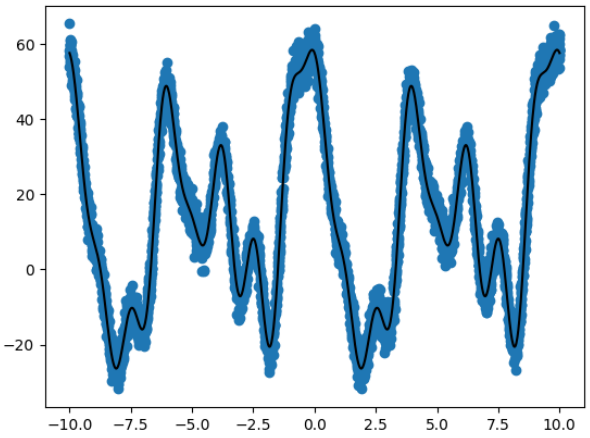
Figure 2: Accuracy and loss as centered random perturbations are added to the nonfinal weight layers of the DNN.

Remark 7.4. In a linear regression problem with noisy Fourier-generated target functions and a fixed feature map, MP-guided pruning recovers the low-rank structure more effectively than L_1 or L_2 regularization alone. Specifically, pruning achieves mean squared error 0.149, compared with 0.416 for L_1, 2.563 for L_2, and 2.574 for no regularization. The pruned spectrum closely matches the ideal low-rank spectrum, whereas L_1 and L_2 leave a visible residual bulk.

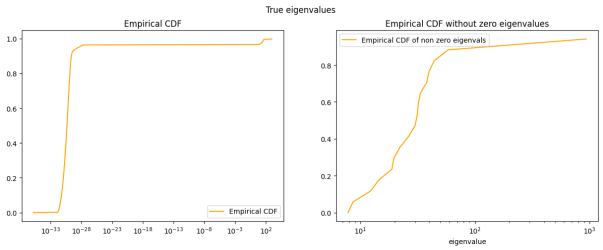
Figure 3 collects the regression illustration: panel a shows one noisy coordinate of the data, panel b shows the ideal low-rank spectrum, and panels c–e compare the L^2, L^1, and pre-pruning spectra.

8 Algorithms for RMT-based pruning diagnostics

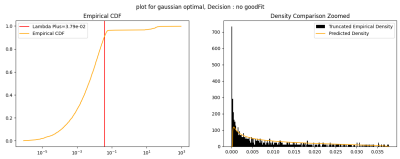
The BEMA edge-fitting algorithm and the MP-fit metric are the building blocks of every RMT-driven protocol in this paper and Main.



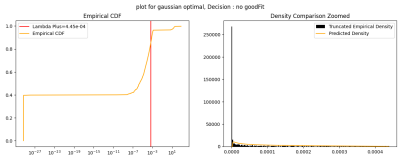
(a) One coordinate of the noisy regression data.



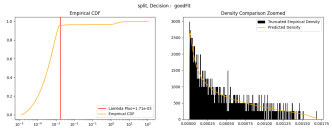
(b) Ideal low-rank spectrum.



(c) L^2-regularized spectrum.



(d) L^1-regularized spectrum.



(e) Spectrum before pruning.

Figure 3: Regression illustration for comparing pruning with more classical regularization.

8.1 BEMA technique for computing λ_+

We outline the BEMA technique for estimating the right MP edge λ_+ of $X = \frac{1}{N} W^\top W$ from its eigenvalue ESD. The full procedure appears in [16]. Below is a streamlined square-matrix version.

Algorithm 1 Computing λ_+ using MP and Tracy–Widom corrections

procedure BEMAEdge($\lambda_1, \ldots, \lambda_N; \alpha, \beta$)
  Pick parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$.
  for each $\alpha N \le k \le (1-\alpha)N$ **do**
    Calculate q_k, the (k/N)-upper quantile of the MP law with $\sigma^2 = 1$ and $c = 1$.
  end for
  Evaluate

$$\widehat{\sigma}^2 = \frac{\sum_{\alpha N \le k \le (1-\alpha)N} q_k \lambda_k}{\sum_{\alpha N \le k \le (1-\alpha)N} q_k^2}.$$

  Let $t_{1-\beta}$ be the $(1-\beta)$-quantile of the Tracy–Widom distribution.
  Output

$$\lambda_+ = \widehat{\sigma}^2 \left[4 + 2^{4/3} t_{1-\beta} N^{-2/3}\right].$$

end procedure

Algorithm 2 Spectral analysis using a BEMA-type fit

procedure SPECTRALFIT($W; \alpha, \beta, \gamma_{\text{fit}}$)
  Set $X = \frac{1}{N} W^\top W$.
  Compute the eigenvalues $\lambda_1, \ldots, \lambda_M$ of X.
  Compute the empirical CDF F_X of the eigenvalue ESD.
  Apply BEMAEDGE to estimate λ_+.
  Let F_X^{MP} be the fitted MP CDF.
  Compute the Kolmogorov-type fit error on the central quantile window:

$$\mu_{\text{fit}} = \max_{i_{\min} \le i \le i_{\max}} \left| F_X(\lambda_i) - F_X^{\text{MP}}(\lambda_i) \right|.$$

  Accept the layer as "sufficiently MP-like" if $\mu_{\text{fit}} \le \gamma_{\text{fit}}$.
end procedure

Algorithm 3 MP-based pruning algorithm for the fully connected examples

Require: Epoch block length ℓ, MP-fit threshold τ, monotone schedule $f(\text{epoch})$, and flags $\text{split}_\ell =$ false for each layer.

Train the DNN for ℓ epochs and set epoch $:= \ell$.
while the training condition is met **do**
  for each layer W_ℓ with $\text{split}_\ell =$ false **do**
    Compute the SVD $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$.
    Form $X_\ell = \frac{1}{N_\ell} W_\ell^\top W_\ell$ and estimate λ_+ via BEMA.
    Set $\sigma_+ := \sqrt{N_\ell \lambda_+}$.
    Test whether the bulk of X_ℓ is well fit by MP.
    if the bulk fits MP **then**
      Remove a fraction $1 - f(\text{epoch})$ of the singular values below σ_+ to obtain Σ'_ℓ.
      Form $W'_\ell = U_\ell \Sigma'_\ell V_\ell^\top$.
      Optionally factor W'_ℓ as $W'_{1,\ell} W'_{2,\ell}$ via $\sqrt{\Sigma'_\ell}$ if that reduces parameters.
    end if
  end for
  Train the DNN for another ℓ epochs and update epoch $:=$ epoch $+ \ell$.
end while

Algorithm 4 Sparsify singular vectors

Require: Layer matrix W_ℓ, threshold θ, fitted singular-value edge σ_+.

Compute the SVD $W_\ell = U\Sigma V^\top$.

for $i = 1, \dots, \min\{N, M\}$ **do**

Compute

$$T_i = \theta \max\left\{\frac{1}{750}, \left(1 - \frac{\Sigma_{ii}}{\sigma_+}\right)_+^{30}\right\}.$$

Sparsify the singular vectors:

$$U_{:,i} \leftarrow \mathrm{Prune}(U_{:,i}, T_i), \qquad V_{:,i} \leftarrow \mathrm{Prune}(V_{:,i}, T_i).$$

end for

Recompose $W'_\ell = U\Sigma V^\top$.

Algorithm 5 Element-wise sparsification

Require: Matrix W_ℓ, threshold ξ.

for each entry $(W_\ell)_{ij}$ **do**

if $|(W_\ell)_{ij}| < \xi$ **then**

Set $(W'_\ell)_{ij} = 0$.

else

Set $(W'_\ell)_{ij} = (W_\ell)_{ij}$.

end if

end for

Return W'_ℓ.

8.2 How well does the ESD of X fit the MP distribution?

Definition 8.1. Let G be a matrix with eigenvalue ESD μ_G when G is positive semidefinite. Its observed cumulative spectral distribution is

$$F_G(a) = \mu_G((-\infty, a]).$$

The MP-fit metric compares the observed cumulative spectral distribution with the fitted MP CDF on a central quantile interval. This focuses the fit test on the bulk and deemphasizes a few possible outliers.

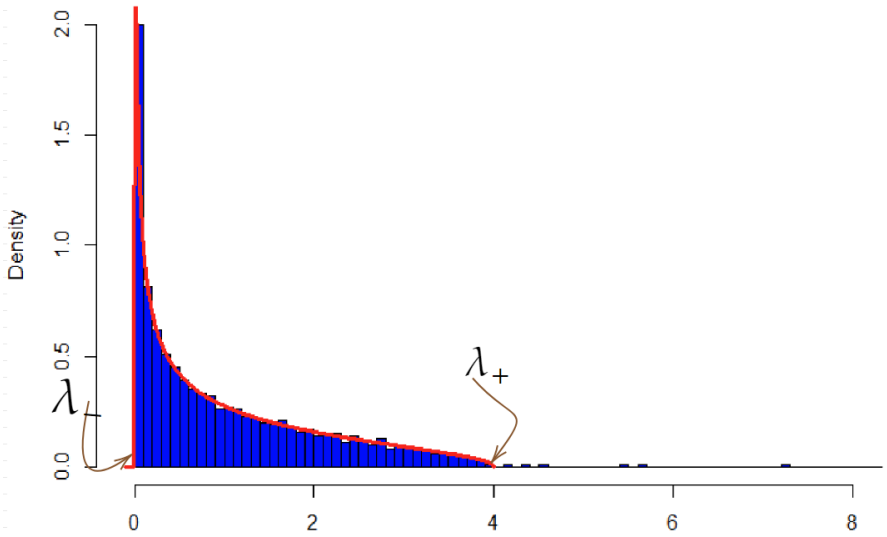
8.3 A BEMA illustration and metric figures

Example 8.2. Let R be an $N \times N$ matrix with i.i.d. $\mathcal{N}(0,1)$ entries, and let S be the deterministic matrix with entries

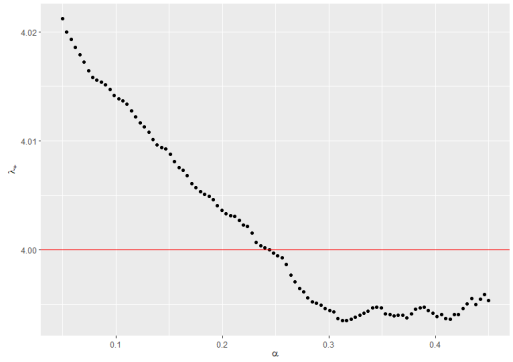
$$S[i,j] = \tan\left(\frac{\pi}{2} + \frac{1}{j+1}\right) + \cos(i)\log(i+j+1) + \sin(j)\cos\left(\frac{i}{j}\right).$$

Set $W = R + S$ and $X = \frac{1}{N}W^\top W$. Figure 4a shows the ESD of X together with the fitted MP distribution produced by BEMA.

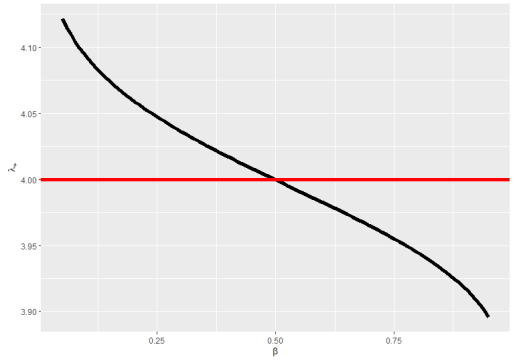
Remark 8.3. The BEMA fit depends on $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$. Figure 4 collects the synthetic BEMA diagnostics: panel a shows the fitted MP bulk in the square unit-variance example, and panels b and c illustrate the dependence on α and β, where the true covariance-scale edge is $\lambda_+ = 4$.



(a) ESD of $X = \frac{1}{N} W^\top W$ together with the fitted MP bulk.



(b) Dependence on α, with $\beta = 0.5$.



(c) Dependence on β, with $\alpha = 0.25$.

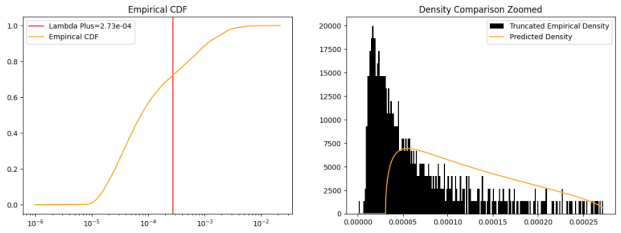
Figure 4: Synthetic BEMA diagnostics: the fitted MP bulk for $X = \frac{1}{N} W^\top W$ and the dependence of the fitted edge λ_+ on α and β.

Heavy-tailed spectra are not ubiquitous in ViTs. Even though ViTs generalize well on ImageNet, their layer ESDs do *not* always exhibit heavy-tailed behavior. In many layers we observe spectra that are well explained by an MP bulk with at most a few outliers, while other layers show more power-law-like decay. This heterogeneity motivates empirical layerwise diagnosis rather than assuming a single universal spectral regime.

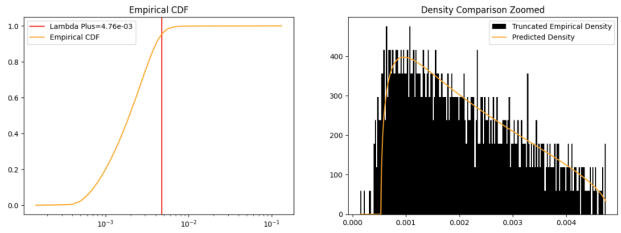
9 Spectral Edge Budgeting and hybrid RMT pruning protocols

This section documents the SEB and SER protocols, the four-regime strategy, the layer-type extension, and the final SEB comparison and validation ledgers. Main-Table 2 (multi-architecture Hybrid Magnitude–SER) is the result of this protocol family.

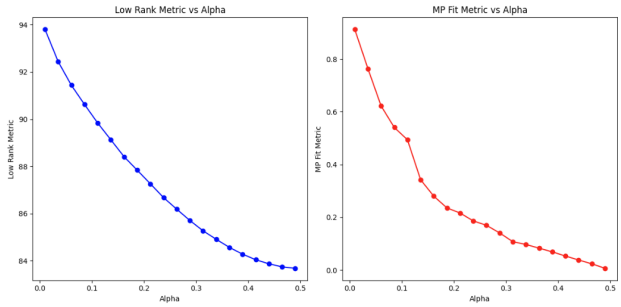
This section records the operational RMT pruning protocols used in Section **??**. The central method is *Spectral Edge Budgeting* (SEB): the fitted Marchenko–Pastur upper edge $\sigma_+(\ell)$ guides the *per-layer pruning budget* inside an otherwise standard global magnitude framework. We also define *Spectral Edge Reallocation* (SER), the practical prune–restore method that over-prunes, ranks a reservoir of removed weights using the same RMT diagnostics, reinserts a fixed budget, and then fine-tunes with the zero



(a) A layer with relatively poor MP fit: MP-fit error 0.74 and bulk fraction 73%.



(b) A layer with strong MP fit: MP-fit error 0.01 and bulk fraction 99.73%.



(c) For the base ViT model: α versus the bulk fraction and MP-fit error.

Figure 5: Layerwise MP diagnostics for the base ViT model. Panels a and b show that some layers are much closer to an MP bulk than others, while panel c shows how the bulk fraction and fit error vary with α.

mask frozen. Finally, we define the Hybrid Magnitude–SER protocol used for broader architecture validation.

The cycle-based classical RMT procedure is a diagnostic baseline, not the final ViT pruning method. It demonstrates that MP-bulk structure is visible in trained ViT layers, but direct singular-vector or singular-value pruning is brittle in modern transformer checkpoints. The final methods use the MP fit to decide *where* magnitude pruning and restoration should occur.

9.1 Spectral Edge Budgeting

Let $W_1, \ldots, W_L$ denote the weight matrices of the target layers. For dense projections, W_ℓ is the layer matrix itself; for a convolutional kernel of shape $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$, we unfold it as a $C_{\text{out}} \times (C_{\text{in}} k_h k_w)$ matrix before fitting the spectrum. Let $\sigma_+(\ell)$ denote the fitted Marchenko–Pastur upper edge for layer ℓ, computed from the eigenvalue distribution of $W_\ell^\top W_\ell / N$ as described in Subsection 8.2. We define the standardized spectral signal

$$z_\ell = \frac{\sigma_+(\ell) - \bar{\sigma}_+}{\text{sd}(\sigma_+)},$$

where $\bar{\sigma}_+$ and $\text{sd}(\sigma_+)$ are the mean and standard deviation of $\{\sigma_+(\ell)\}_{\ell=1}^{L}$ across all target layers. A high z_ℓ indicates that the layer's spectral bulk extends farther, so a larger share of its singular values falls inside the MP-predicted regime and the layer has more spectral redundancy available for pruning.

Given a target global sparsity $s \in (0, 1)$, modulation strength $\beta > 0$, and decay endpoint $s_{\text{d}} > 0$, we define a per-layer pruning weight

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta\, d(s)\, z_\ell\Big), \tag{7}$$

where the decay factor

$$d(s) = \big[\, 1 - s/s_{\mathrm{d}} \,\big]_+ \tag{8}$$

ensures that the modulation attenuates toward uniform magnitude pruning as the sparsity target approaches s_{d}. The floor at 0.1 prevents any layer from being fully exempt.

Pruning proceeds by assigning each nonzero parameter $(W_\ell)_{ij}$ a global score

$$r_{ij}^{(\ell)} = \frac{|(W_\ell)_{ij}|}{w_\ell(s)},$$

and removing the parameters with the smallest scores until the target sparsity s is reached. Equivalently, if the global score cutoff is q, then layer ℓ is thresholded in magnitude at $q\, w_\ell(s)$. Thus $w_\ell > 1$ raises the effective magnitude threshold and causes more parameters to be pruned from that layer; conversely, $w_\ell < 1$ protects the layer. At $\beta = 0$ (or when $s \geq s_{\mathrm{d}}$), all weights equal 1 and the method reduces to plain global magnitude pruning.

9.2 Layer-type extension

A ViT contains two functionally distinct families of dense layers: *attention projections* (query–key–value and output) and *MLP layers* (the two feed-forward projections per block). We extend the modulation (7) by allowing different modulation strengths for each family:

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta_{\mathrm{type}(\ell)}\, d(s)\, z_\ell\Big), \qquad \beta_{\mathrm{type}(\ell)} = \begin{cases} \beta_{\mathrm{attn}}, & \ell \in \mathcal{L}_{\mathrm{attn}}, \\ \beta_{\mathrm{mlp}}, & \ell \in \mathcal{L}_{\mathrm{mlp}}, \\ \beta, & \text{otherwise}, \end{cases} \tag{9}$$

where $\mathcal{L}_{\mathrm{attn}}$ and $\mathcal{L}_{\mathrm{mlp}}$ partition the transformer layers by type. The z-scores remain globally computed across all layers, preserving the cross-type spectral ranking; only the *strength* of the modulation varies by type.

9.3 Alternative signal: the power-law exponent α

The heavy-tailed self-regularization (HT-SR) framework of Martin and Mahoney [23, 22] characterizes each layer by the power-law exponent α_ℓ of the tail of its eigenvalue distribution, originally estimated via maximum-likelihood power-law fitting on the squared singular values; AlphaPruning [21] subsequently introduces a Hill-estimator variant ("PL Alpha Hill") as a more robust replacement for the MLE. Lower α_ℓ indicates a more heavy-tailed (better-trained, more structured) layer; higher α_ℓ indicates a more random-like layer. This metric is the basis of the AlphaPruning method [21], which assigns per-layer sparsity in proportion to α_ℓ.

We use α_ℓ as a drop-in replacement for $\sigma_+(\ell)$ in the budget-modulation framework (7), with the standardized signal

$$z_\ell^{\alpha} = \frac{\alpha_\ell - \bar{\alpha}}{\mathrm{sd}(\alpha)}.$$

High z_ℓ^{α} indicates a more random-like layer that should absorb more pruning—the same direction as σ_+.

Crucially, α and σ_+ are only weakly correlated ($r = 0.37$ across the 50 target layers of ViT-B/16). They capture complementary aspects of the spectral structure: σ_+ measures the extent of the MP bulk (how much of the spectrum is noise-like), while α measures the tail decay rate (how well-trained the layer is overall). This low correlation suggests that each signal may be informative in a different sparsity regime.

9.4 Four-regime strategy

In these sweeps, α gives higher top-1 than σ_+ at very low sparsity ($s \leq 0.20$), while σ_+ gives higher top-1 at moderate and high sparsity ($s \geq 0.25$). This motivates a four-regime strategy that selects the signal and hyperparameters per sparsity range:

1. **Very low sparsity** ($s \leq 0.20$): α-budget with $\beta = 0.50$, $s_{\mathrm{d}} = 0.30$. The power-law exponent provides mild but positive modulation where σ_+ is indistinguishable from magnitude pruning.
2. **Low sparsity** ($0.20 < s \leq 0.40$): σ_+-budget with $\beta = 1.00$, $s_{\mathrm{d}} = 0.50$.
3. **Moderate sparsity** ($0.40 < s \leq 0.55$): σ_+-budget with $\beta = 1.25$, $s_{\mathrm{d}} = 0.70$.
4. **High sparsity** ($s > 0.55$): Layer-type σ_+-budget with $\beta_{\mathrm{attn}} = 1.00$, $\beta_{\mathrm{mlp}} = 2.00$, $s_{\mathrm{d}} = 0.85$.

The transition from α to σ_+ at $s = 0.20$ reflects an empirical regime shift in these sweeps: at very low sparsity the binding constraint is layer-level training quality (captured by α), whereas at moderate-to-high sparsity the binding constraint is spectral redundancy (captured by σ_+).

9.5 Hyperparameter space and optimization

The method has three base hyperparameters $(\beta, s_{\mathrm{d}}, p)$ for uniform modulation (where $d(s) = [1 - s/s_{\mathrm{d}}]_+^p$ generalizes (8)), plus the optional layer-type extension $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ and the choice of signal (σ_+ or α). We performed a systematic grid search totalling over 1,000 evaluations on the ViT-B/16 architecture over five successive sweeps:

1. **Signal comparison** (175 cells). We compared four per-layer signals—σ_+, $\sigma_+/\|W\|_F$, the excess kurtosis of $|W|$, and the coefficient of variation of $|W|$—each with $\beta \in \{0.25, 0.50, 1.00\}$, $s_{\mathrm{d}} \in \{0.50, 0.70\}$, across 7 target sparsities. Only the MP edge σ_+ produced consistent improvements over magnitude pruning; the other three signals either matched or degraded performance at every configuration tested.
2. **Decay shape and extended reach** (108 cells). Fixing the signal to σ_+, we tested decay exponents $p \in \{1, 2, 3\}$ and extended the decay endpoint to $s_{\mathrm{d}} \in \{0.70, 0.85\}$. Nonlinear decay ($p > 1$) proved uniformly harmful: at $s = 0.55$, quadratic decay ($p = 2$) lost 28 percentage points relative to linear ($p = 1$). Extending s_{d} from 0.70 to 0.85 yielded large gains at $s \geq 0.60$.
3. **Fine grid refinement** (206 cells). We explored $\beta \in \{0.75, \ldots, 1.75\}$, $s_{\mathrm{d}} \in \{0.70, \ldots, 1.05\}$, and the sub-linear regime $p \in \{0.5, 0.75, 1.0\}$. The surface is a broad diagonal ridge in (β, s_{d})-space: stronger modulation compensates for faster decay and vice versa. Sub-linear decay ($p < 1$) did not improve over linear.
4. **Definitive evaluation and layer-type extension** (434 cells). We re-evaluated the top 14 uniform-β methods at 250-batch full precision across 11 sparsities (Phase 1, 154 cells), performed an ultra-fine 9×5 grid of (β, s_{d}) at the cliff region (Phase 2, 180 cells), and ran a full 5×5 grid of $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ at 4 sparsities (Phase 3, 100 cells).
5. **Alpha signal sweep** (77 cells). We replaced σ_+ with the Hill-estimated power-law exponent α_ℓ and tested $\beta \in \{0.25, 0.50, 1.00\}$, $s_{\mathrm{d}} \in \{0.25, 0.30, 0.50, 0.70\}$ across 11 sparsities. The α signal reports higher top-1 than both magnitude and σ_+ at $s \leq 0.20$ but collapses at $s \geq 0.30$, supporting the complementary-regime selection used below.

All evaluations used a fixed subset of 2,000 images from the ILSVRC-2012 validation set (250 batches of 8) unless otherwise noted; definitive results in Phase 1 used the same subset at 250-batch evaluation for reduced noise (± 0.5 pp).

9.6 Results: uniform modulation

Table 6 reports the 250-batch top-1 accuracy for the sweep-selected uniform-β configuration at each target sparsity, compared with global magnitude pruning. The selected parameters shift systematically: at moderate sparsity ($s \leq 0.55$), $s_{\mathrm{d}} = 0.70$ with $\beta = 1.25$ suffices; at high sparsity ($s \geq 0.60$), the decay must extend further ($s_{\mathrm{d}} \geq 0.85$) so that the σ_+ signal remains active through the critical region.

Table 6: Sweep-selected uniform-β σ_+-budget method versus global magnitude pruning on ViT-B/16 (no retraining). All results at 250-batch evaluation. The Δ column reports the improvement over magnitude pruning.

Sparsity s	Selected (β, s_{d})	Top-1 (%)	Magnitude (%)	Δ (pp)
0.30	(1.00, 0.50)	84.90	84.25	+0.65
0.40	(1.00, 0.70)	82.80	82.75	+0.05
0.50	(1.25, 0.70)	79.30	76.20	+3.10
0.55	(1.25, 0.70)	72.85	67.70	+5.15
0.60	(1.25, 0.85)	59.85	45.00	+14.85
0.625	(1.30, 0.90)	48.45	26.60	+21.85
0.65	(1.30, 0.90)	33.25	10.75	+22.50
0.675	(1.30, 0.95)	17.35	2.55	+14.80
0.70	(1.50, 0.95)	5.80	0.70	+5.10

Key findings from the uniform-β sweeps.

1. *Linear decay was the selected schedule.* Across all 923 cells, the linear schedule $p = 1$ in (8) matched or exceeded the tested higher-order decay schedules ($p \geq 2$). Quadratic and cubic decay cause the modulation to retreat too quickly at moderate sparsity, collapsing to magnitude pruning in the regime where the spectral signal contributes in the sweep. Sub-linear decay ($p < 1$) over-modulates, hitting the $w_\ell = 0.1$ floor for too many layers.

2. *The surface is a diagonal ridge.* An ultra-fine 9×5 grid at the cliff ($s \in \{0.60, 0.625, 0.65, 0.675\}$) reveals that many (β, s_{d}) pairs give nearly identical accuracy. For example, at $s = 0.65$, the configurations $(\beta = 1.25, s_{\mathrm{d}} = 0.94)$, $(\beta = 1.35, s_{\mathrm{d}} = 0.91)$, and $(\beta = 1.45, s_{\mathrm{d}} = 0.88)$ all achieve $\approx 33.6\%$ top-1. The effective control variable is the product $\beta \cdot d(s)$, the modulation strength at the operating point.

3. *The gain is largest at the accuracy cliff.* Below $s = 0.50$, σ_+-modulation provides modest gains ($\leq$3 pp). The improvement grows rapidly beyond $s = 0.55$ and peaks at +22.5 pp near $s = 0.65$, precisely where uniform magnitude pruning exhausts the "safe" parameters and begins removing structurally important weights. The σ_+ signal identifies layers with residual capacity that magnitude pruning alone cannot exploit.

9.7 Results: layer-type modulation

Table 7 reports the 150-batch top-1 accuracy for the full 5×5 grid of $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ at $s = 0.65$, with $s_{\mathrm{d}} = 0.85$ and $p = 1$ held fixed.

The sweep favors asymmetric modulation: $\beta_{\mathrm{attn}} = 1.00$ (minimal spectral modulation for attention layers) and $\beta_{\mathrm{mlp}} = 2.00$ (strong spectral modulation for MLP layers). This configuration achieves 39.75% top-1 at $s = 0.65$, gaining +6.67 pp over the uniform-β diagonal value (33.08%) and +29.00

Table 7: Layer-type modulation at $s = 0.65$: top-1 accuracy (%) for each $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ pair. The uniform-β diagonal is shown in bold. The largest value in this grid is at $(\beta_{\text{attn}}, \beta_{\text{mlp}}) = (1.00, 2.00)$, corresponding to near-uniform magnitude pruning for attention and stronger σ_+-guided redistribution for MLP layers.

	β_{mlp}				
β_{attn}	1.00	1.25	1.50	1.75	2.00
1.00	27.50	32.75	36.00	36.17	**39.75**
1.25	27.17	32.33	34.33	37.00	38.00
1.50	24.25	30.75	**33.08**	35.25	36.92
1.75	22.58	27.67	30.42	32.17	33.67
2.00	19.00	24.92	27.58	28.67	30.25

pp over magnitude pruning (10.75%). The layer-type gain is consistent across sparsities: +3.42 pp at $s = 0.60$, +4.92 pp at $s = 0.625$, and +5.92 pp at $s = 0.675$.

Interpretation. At high sparsity, attention projection matrices appear well served by magnitude pruning alone: the σ_+ signal provides no additional guidance and can even mislead the budget allocation for these layers. MLP layers, by contrast, exhibit within-family heterogeneity in spectral structure that the σ_+ signal captures well. Applying strong budget modulation only to MLP layers lets magnitude pruning handle attention while directing spectral guidance to the layers where it provides discriminative power.

9.8 SEB final comparison

The selected hyperparameters vary systematically with sparsity (Table 6): at low sparsity, gentle modulation with a short decay suffices; at high sparsity, the layer-type extension with strong MLP modulation dominates. This motivates the final SEB strategy, which uses the sweep-selected configuration per sparsity range. We define three regimes based on the empirical accuracy landscape:

1. **Low sparsity** ($s < 0.30$): The model retains most of its capacity and magnitude pruning remains close to the dense baseline. At this sparsity level, the per-layer redundancy distribution is not yet a binding constraint: across all 923 configurations tested, no σ_+-modulated variant consistently exceeds global magnitude pruning. Accordingly, the reported protocol uses magnitude pruning without spectral modulation for $s < 0.30$, and begins applying the σ_+-budget from $s \geq 0.30$ onward, where ($\beta = 1.00$, $s_{\text{d}} = 0.50$) yields +0.65 pp over magnitude.

2. **Moderate sparsity** ($0.40 < s \leq 0.55$): Magnitude pruning begins to lose accuracy rapidly. Uniform modulation with $\beta = 1.25$, $s_{\text{d}} = 0.70$ recaptures 1–5 percentage points by directing pruning toward spectrally redundant layers.

3. **High sparsity** ($s > 0.55$): The accuracy cliff. The layer-type extension with $\beta_{\text{attn}} = 1.00$, $\beta_{\text{mlp}} = 2.00$, $s_{\text{d}} = 0.85$ yields gains of 6–29 percentage points over magnitude pruning by preserving attention layers and concentrating spectral redistribution on MLP layers.

Table 8 compares SEB against global magnitude pruning on ViT-B/16, evaluated at every 5% sparsity increment from 5% to 75%. No retraining or fine-tuning is applied; all accuracy values are from a single forward pass on a 2,000-image subset of the ILSVRC-2012 validation set.

Table 8: Top-1 accuracy (%) on ViT-B/16 for global magnitude pruning versus SEB. At low sparsity ($s \leq 0.40$), the two methods are within the evaluation noise band; the reported σ_+-budget gap increases from $s \geq 0.45$ and reaches +28.60 pp at $s = 0.65$.

Sparsity	Regime	Magnitude (%)	σ_+-budget (%)	Δ (pp)
30%	Low	84.25	84.90	+0.65
35%	Low	83.85	83.75	−0.10
40%	Low	82.75	82.55	−0.20
45%	Mid	80.00	81.30	+1.30
50%	Mid	76.20	79.30	+3.10
55%	Mid	67.70	72.85	+5.15
60%	High	45.00	61.80	+16.80
65%	High	10.75	39.35	+28.60
70%	High	0.70	6.25	+5.55
75%	High	0.10	0.25	+0.15

In this sweep, the σ_+-budget method does not reduce top-1 by more than the 2,000-image evaluation noise band at low sparsity: the lowest gap is −0.75 pp at $s = 0.20$. This is consistent with the theoretical expectation that low-sparsity masks have small propagated effects, so the per-layer budget redistribution is a second-order effect. At higher sparsity the reported gap is largest at $s = 0.65$ (39.35% versus 10.75%).

9.9 Singular-vector sparsification as a preprocessing step

The spectral analysis in previous sections operates on the *singular values* of each weight matrix. A natural question is whether sparsifying the *singular vectors* — zeroing small components of $\mathbf{U}$ and $\mathbf{V}$ before magnitude pruning — can further improve accuracy by removing noise in the rank-one directions themselves.

We test this with a Haar-distribution test: for each weight matrix $W \in \mathbb{R}^{M \times N}$, we compute the SVD $W = U\Sigma V^\top$, then for each singular vector column, compute a z-score against the Haar (uniform-on-the-sphere) null distribution. Components with $|z| < z_{\text{base}}$ are zeroed, effectively denoising the singular subspaces before the weight matrix is reconstructed. The threshold is modulated per layer by σ_+ with power $p = 3$, following the same principle as the budget allocation: layers with larger spectral bulk have more noise in their singular vectors.

We sweep $z_{\text{base}} \in \{0.3, 0.5, 0.7, 1.0\}$ and apply the denoised model as a preprocessing step before standard global magnitude pruning. Table 9 reports the results.

The effect is marginal: the largest SV gain over the no-SV baseline is 1.65 pp (at $s = 0.65$, $z = 1.0$), and at most sparsity levels the differences are ≤ 0.25 pp — within the noise of a 2,000-image evaluation. This negative result suggests that, in these sweeps, the useful pruning signal for ViT-B/16 weight matrices was better captured by the *magnitudes* of the singular values (i.e., the spectral distribution), not by the *directions* of the singular vectors. The Marchenko–Pastur edge σ_+ captures the relevant signal; further denoising of the subspace bases does not provide additional compression benefit at any sparsity level tested.

Table 9: Top-1 accuracy (%) after Haar singular-vector sparsification (SV) followed by global magnitude pruning, compared to magnitude pruning alone. SV preprocessing yields small improvements at low sparsity (+0.25 pp at $s = 0.05$ for $z = 0.3$) and moderate sparsity (+0.90 pp at $s = 0.60$ for $z = 0.5$), but all gains are within or near the evaluation noise band.

Sparsity	No SV (%)	$z = 0.3$ (%)	$z = 0.5$ (%)	$z = 0.7$ (%)	$z = 1.0$ (%)
5%	86.05	**86.30**	85.95	85.80	85.90
10%	**86.10**	86.00	85.95	85.90	85.90
15%	85.70	85.75	85.70	85.85	**85.90**
20%	**85.70**	85.55	**85.75**	85.50	85.40
25%	84.85	84.90	**84.95**	**84.95**	84.60
30%	84.25	84.25	**84.45**	**84.45**	84.25
60%	45.00	45.50	**45.90**	45.60	45.55
65%	10.75	11.60	11.10	11.75	**12.40**
70%	**0.70**	**0.70**	1.00	1.10	1.30

9.10 Full validation-set evaluation of SEB

The sweep tables in this appendix report accuracy from a 2,000-image validation subset, selected for speed during the hyperparameter sweep. After selecting the final configuration, we re-evaluate on the *complete* 50,000-image ILSVRC-2012 validation set, matching the standard evaluation protocol used to report the baseline accuracy of 85.11%. The checkpoint is `vit_base_patch16_224.augreg2_in21k_ft_in1k`. For the reported SEB row, we apply Haar singular-vector sparsification ($z = 0.5$, $p = 3$) as a preprocessing step followed by the final SEB budget; this corresponds to composing the modulation of Section 9.8 with the singular-vector step of Section 9.9.

The full-validation evaluation confirms the qualitative picture from the 2,000-image sweep (Table 8): at low sparsity the two methods are within measurement noise, while the SEB advantage grows sharply once magnitude pruning begins to collapse. The crossover begins at $s = 0.45$ (+0.49 pp) and the gap widens monotonically to +28.95 pp at $s = 0.65$, where magnitude pruning has dropped to 9.97% top-1 while SEB retains 38.92%. Beyond $s = 0.70$, neither method is practically useful without fine-tuning, and the differences compress accordingly. These are the final sweep-selected no-fine-tuning spectral-guided values; SER extends the same spectral idea to a fine-tuned prune–restore pipeline.

9.11 Spectral Edge Reallocation

SEB is a one-shot budget allocation method. SER turns the same spectral signal into a practical prune–restore pipeline. For a target sparsity s, the method first prunes past the target, producing a reservoir $\mathcal{R}$ of removed entries. During a short selection phase, only the removed entries are trainable; the method records their absolute movement from the pre-selection reservoir value. An entry (i, j) in layer ℓ is then ranked by

$$|\Delta_{ij}^{(\ell)}|\, \rho_\ell(s),$$

where $\rho_\ell(s)$ is the RMT restoration multiplier obtained from the same layerwise spectral weights used for pruning. Thus entries that move strongly during the selection phase and belong to spectrally protected layers receive higher restoration priority, while entries from layers with larger MP-bulk pruning capacity are less likely to be restored. A fixed restoration budget is reinserted, and the final mask is adjusted so that the realized post-reinsertion sparsity is exactly s. The surviving weights are then fine-tuned for a short schedule with the zero mask frozen.

Table 10: Full 50,000-image validation evaluation of SEB with Haar singular-vector sparsification ($z = 0.5$, $p = 3$) versus global magnitude pruning on ViT-B/16. Baseline top-1 accuracy: 85.11%. No fine-tuning or retraining is applied. Bold entries mark sparsities at which SEB is higher than magnitude by ≥ 1 pp.

Sparsity s	Regime	Magnitude (%)	SEB (%)	Δ (pp)
5%	Low	85.09	85.08	−0.01
10%	Low	85.06	85.02	−0.04
15%	Low	85.02	84.85	−0.17
20%	Low	84.89	84.56	−0.34
25%	Low-mid	84.58	84.53	−0.05
30%	Low-mid	84.07	84.30	+0.23
35%	Low-mid	83.45	83.66	+0.21
40%	Low-mid	82.23	82.24	+0.01
45%	Mid	80.20	80.70	+0.49
50%	Mid	76.67	**78.55**	+1.88
55%	Mid	68.41	**72.68**	+4.28
60%	High	46.42	**61.57**	+15.15
65%	High	9.97	**38.92**	+28.95
70%	High	0.82	**6.11**	+5.30
75%	High	0.20	0.35	+0.15

The connection to the theory is the same as for SEB, but applied twice. The pruning stage uses MP information to allocate perturbation budget across layers; the reallocation stage uses the same information to choose which removed perturbations were too costly and should be restored. The deterministic certificates of Section **??** apply after the mask is fixed, while the MP diagnostics supply a data-independent ranking rule for finding masks that are more likely to satisfy those certificates.

9.12 Hybrid Magnitude–SER

The sweep above shows that RMT modulation is not needed at very low sparsity: below about 20%, global magnitude pruning is already within the noise band of the spectral rules tested here. The hybrid method therefore uses exact global magnitude pruning for $s \leq 0.20$, saves those checkpoints, and starts SER only for the continuation $s > 0.20$. Its purpose is conservative: preserve the low-sparsity behavior of magnitude pruning, then use RMT only in the regime where layerwise redundancy becomes the limiting factor.

9.13 Checkpoint validation ledger

The main tables report rounded top-1 accuracy values from the archived validation files associated with the saved checkpoints. Most entries come directly from `results.json` or `finetune_results.json`; where a later restart overwrote a partial JSON ledger, the local archived stdout validation line and the saved checkpoint are used. When both a prefix file and the integrated run file contain the same exact-magnitude prefix checkpoint, the integrated run file is used in the ledger; prefix-only rows are included only when no later integrated validation file exists. Entries are post-fine-tuning ImageNet-1k validation top-1 accuracy in percent. Stopped exploratory rows with no validated nonzero checkpoint are omitted from Table 16.

Table 11: Checkpoint-level validation ledger for the Hybrid Magnitude–SER multi-architecture results. These are the values used in Table 16.

Architecture/checkpoint	Dense	Validated sparsity–top-1 pairs
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
ViT-B/16/384 `augreg_in21k_ft_in1k`	86.01	0.05 : 86.05, 0.10 : 86.09, 0.15 : 86.07, 0.20 : 85.99, 0.25 : 86.00, 0.30 : 85.90, 0.35 : 85.81, 0.40 : 85.78, 0.45 : 85.48, 0.50 : 85.15, 0.55 : 84.64, 0.60 : 83.35, 0.65 : 82.50, 0.70 : 81.33
ViT-Large/16 `augreg_in21k_ft_in1k`	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 84.98, 0.30 : 85.32, 0.35 : 85.64, 0.40 : 85.04, 0.45 : 85.08, 0.50 : 84.52, 0.55 : 84.34, 0.60 : 83.81, 0.65 : 83.02, 0.70 : 82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	0.05 : 72.41, 0.10 : 72.38, 0.15 : 72.18, 0.20 : 71.86, 0.25 : 71.99, 0.30 : 71.75, 0.35 : 71.35, 0.40 : 70.66, 0.45 : 68.97, 0.50 : 67.74, 0.55 : 65.91, 0.60 : 61.17, 0.65 : 55.58, 0.70 : 52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	0.05 : 79.82, 0.10 : 79.78, 0.15 : 79.62, 0.20 : 79.37, 0.25 : 79.31, 0.30 : 79.21, 0.35 : 79.00, 0.40 : 78.61, 0.45 : 78.06, 0.50 : 77.22, 0.55 : 76.25, 0.60 : 74.14, 0.65 : 72.05, 0.70 : 68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	0.05 : 81.76, 0.10 : 81.70, 0.15 : 81.58, 0.20 : 81.46, 0.25 : 81.42, 0.30 : 81.28, 0.35 : 81.17, 0.40 : 80.99, 0.45 : 80.62, 0.50 : 80.12, 0.55 : 79.49, 0.60 : 78.16, 0.65 : 76.72, 0.70 : 74.90
Swin-Tiny `patch4_window7_224.ms_in1k`	81.20	0.05 : 81.32, 0.10 : 81.28, 0.15 : 81.25, 0.20 : 81.05, 0.25 : 81.09, 0.30 : 80.91, 0.35 : 80.78, 0.40 : 80.37, 0.45 : 80.14, 0.50 : 79.80, 0.55 : 78.98, 0.60 : 78.06, 0.65 : 76.64, 0.70 : 74.47
ConvNeXt-Base `fb_in22k_ft_in1k`	85.84	0.05 : 85.72, 0.10 : 85.58, 0.15 : 85.51, 0.20 : 85.26, 0.25 : 85.35, 0.30 : 85.39, 0.35 : 85.26, 0.40 : 85.04, 0.45 : 84.94, 0.50 : 84.80, 0.55 : 84.09, 0.60 : 83.70, 0.65 : 83.11, 0.70 : 81.21
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ResNet50d `ra2_in1k`	80.55	0.05 : 79.90, 0.10 : 80.01, 0.15 : 80.02, 0.20 : 80.12, 0.25 : 80.09, 0.30 : 80.12, 0.35 : 80.06, 0.40 : 79.90, 0.45 : 79.48, 0.50 : 79.16, 0.55 : 78.93, 0.60 : 77.93, 0.65 : 77.25, 0.70 : 76.32
ResNet101d `ra2_in1k`	82.26	0.05 : 82.06, 0.10 : 82.05, 0.15 : 82.15, 0.20 : 82.15, 0.25 : 82.06, 0.30 : 82.07, 0.35 : 81.93, 0.40 : 81.97, 0.45 : 81.61, 0.50 : 81.56, 0.55 : 81.47, 0.60 : 80.45, 0.65 : 80.15, 0.70 : 79.82
ResNet50 `tv_in1k`	76.13	0.05 : 75.85, 0.10 : 75.33, 0.15 : 75.72, 0.20 : 76.07, 0.25 : 76.17, 0.30 : 76.13, 0.35 : 76.13, 0.40 : 76.14, 0.45 : 75.75, 0.50 : 75.76, 0.55 : 75.70, 0.60 : 74.83, 0.65 : 74.62, 0.70 : 74.15
ResNet34 `tv_in1k`	73.28	0.05 : 73.00, 0.10 : 72.67, 0.15 : 72.39, 0.20 : 72.62, 0.25 : 73.06, 0.30 : 73.09, 0.35 : 73.16, 0.40 : 73.16, 0.45 : 73.00, 0.50 : 72.88, 0.55 : 72.74, 0.60 : 72.12, 0.65 : 71.59, 0.70 : 71.44
ResNet18 `tv_in1k`	69.76	0.05 : 69.73, 0.10 : 69.52, 0.15 : 69.18, 0.20 : 68.94, 0.25 : 69.02, 0.30 : 69.28, 0.35 : 69.50, 0.40 : 69.50, 0.45 : 69.39, 0.50 : 69.12, 0.55 : 69.00, 0.60 : 68.31, 0.65 : 67.82, 0.70 : 67.23
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	0.05 : 85.04, 0.10 : 84.94, 0.15 : 84.76, 0.20 : 84.39, 0.25 : 77.95, 0.30 : 83.12, 0.35 : 82.15, 0.40 : 82.37, 0.45 : 82.29, 0.50 : 82.25, 0.55 : 82.00, 0.60 : 80.03, 0.65 : 80.55, 0.70 : 78.96

9.14 Classical RMT baseline

The "Classical RMT" entry in Table **??** refers to the cycle-based procedure: fit an MP edge in each layer, treat sub-edge singular components as bulk-like, sparsify small singular-vector coordinates, and then prune coefficients directly. This method is conceptually close to SVD compression and RMT denoising work [28, 30], but it is not the final ViT method in this paper. Its role is to establish that MP structure is present and exploitable; the stronger SEB, SER, and Hybrid Magnitude–SER protocols use the MP edge as a layerwise allocation signal rather than as a hard instruction to delete all sub-edge spectral components.

10 CAST: certificate-aware 2:4 + ToMe conversion

This section gives the 2:4*+ToMe CAST pipeline that underlies the deployable ViT-B/16 row in* Main*. The Stage-1 cost and per-row argmin are the special* 2:4 *case of the general CAST-2E k:n projection described in §3.*

CAST (**C**ertificate-**A**ware **S**parse-**T**oken conversion) is the procedure used to produce the second row of Table 17. This section records the algorithmic details, the connections to the deterministic certificate framework of Section **??**, and the role played by the RMT/SER pipeline that supplies CAST's input checkpoint.

Table 12: Downloaded threshold and restart validation ledgers saved locally. These rows give the provenance for drop-threshold and restart trajectories, including several rows consolidated into Table 16.

Run	Dense	Validated sparsity–top-1 pairs
ViT-Large/16 seeded drop-threshold audit run	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 85.58, 0.30 : 85.42, 0.35 : 85.22, 0.40 : 84.71
ConvNeXtV2-Base drop-threshold ledger	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ViT-Small/16 archived partial ledger	81.40	0.05 : 81.14, 0.10 : 81.05, 0.15 : 80.91, 0.20 : 80.71, 0.25 : 80.11, 0.30 : 80.29, 0.35 : 79.88, 0.40 : 79.29, 0.45 : 77.99, 0.50 : 76.54
DeiT-Base restart ledger	81.80	0.05 : 81.79, 0.10 : 81.69, 0.15 : 81.54, 0.20 : 81.44, 0.25 : 81.25, 0.30 : 81.35, 0.35 : 81.18, 0.40 : 80.95, 0.45 : 80.68, 0.50 : 80.12, 0.55 : 79.48, 0.60 : 78.24, 0.65 : 76.70, 0.70 : 74.78
Swin-Tiny restart ledger	81.20	0.05 : 81.32, 0.10 : 81.29, 0.15 : 81.26, 0.20 : 81.05, 0.25 : 80.77, 0.30 : 80.47, 0.35 : 80.58, 0.40 : 80.43, 0.45 : 80.11, 0.50 : 79.80, 0.55 : 78.94, 0.60 : 78.03, 0.65 : 76.53, 0.70 : 74.31

Table 13: Auxiliary ViT-B/16 checkpoint validations archived with the main runs. These rows document additional RMT/SER sweeps and exact-magnitude-to-SER variants; the Hybrid Magnitude–SER row is the one reported in Tables **??** and 16.

Archived ViT-B/16 run	Validated sparsity–top-1 pairs
SER, one-epoch selection sweep	0.05 : 85.17, 0.10 : 85.07, 0.15 : 85.01, 0.20 : 84.93, 0.25 : 84.86
SER, two-epoch selection sweep	0.05 : 85.18, 0.10 : 85.12, 0.15 : 84.98, 0.20 : 84.91, 0.25 : 84.84, 0.30 : 84.54, 0.35 : 84.36, 0.40 : 83.85, 0.45 : 83.25, 0.50 : 82.19, 0.55 : 81.03, 0.60 : 78.12, 0.65 : 73.80, 0.70 : 67.00
Hybrid Magnitude–SER, exact-magnitude prefix plus SER continuation	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
Hybrid Magnitude–SER, alternate low-sparsity run	0.05 : 85.02, 0.10 : 84.96, 0.15 : 84.96, 0.20 : 84.77, 0.25 : 84.67, 0.30 : 84.70

10.1 Inputs and stages

CAST takes three inputs:

1. A Hybrid Magnitude–SER sparse checkpoint at sparsity $s = 0.35$, produced by the SEB pipeline of Appendix 9.12. The unstructured mask of this checkpoint is denoted M^{SER}; its construction uses the per-layer Marchenko–Pastur edge signal $\sigma_+(\ell)$ (Appendix 9.1, Eq. (7)) to allocate per-layer pruning weights, so the input to CAST is already a product of the RMT-based pipeline.

2. The dense pretrained ViT-B/16 (`vit_base_patch16_224.augreg2_in21k_ft_in1k`, top-1 85.11%). Used both as the source of restored slot values and as the distillation teacher.

3. A small calibration set $\mathcal{C} \subset \mathcal{X}_{\mathrm{train}}$ ($|\mathcal{C}| = 256$ images sampled deterministically from the ImageNet train shards; no validation-set images are used).

CAST runs in two stages:

Stage 1 — Calibration-only mask minimization (zero gradients). For each compatible Linear layer ℓ (in-channel dimension divisible by 4; we exclude the classification head), capture the input activations entering ℓ over $\mathcal{C}$, then solve a small discrete optimization problem per output-row, per input-group of 4 weights. Materialize the chosen weights (taking the SER value at SER-kept slots and the dense pretrained value at slots restored within a 2:4 group). Save the resulting student state dict and the per-layer 2:4 masks.

Stage 2 — Frozen-mask distillation (3 epochs). Apply ToMe $r = 8$ [7] to the model, then fine-tune for exactly three epochs of distillation against the dense teacher with the Stage-1 mask frozen.

The "2:4 budget" that motivates the design is hardware: NVIDIA Sparse Tensor Core 2:4 [24] pays for the 2-of-4 pattern regardless of whether the two kept slots are numerically nonzero, so once a layer is in 2:4 form, restoring an additional non-zero entry inside an existing 4-tuple is free in sparse-execution FLOPs.

10.2 Stage 1: per-row 2:4 search with free restoration

Fix a Linear layer ℓ with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $d_{\text{in}} = 4G_\ell$. Reshape the rows of W into contiguous 4-tuples: write $W_{i,g,k}$ for the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [4]$. Let $W^{\text{SER}}_{i,g,k}$ denote the SER input checkpoint, $W^{\text{dense}}_{i,g,k}$ the dense pretrained checkpoint, and $M^{\text{SER}}_{i,g,k} = \mathbf{1}\{W^{\text{SER}}_{i,g,k} \neq 0\} \in \{0,1\}$.

For each (i,g) we search over the six 2-of-4 keep patterns

$$\mathcal{M}_{2:4} = \big\{m \in \{0,1\}^4 : |m|_0 = 2\big\}, \qquad |\mathcal{M}_{2:4}| = \binom{4}{2} = 6.$$

For a chosen pattern m we materialize the slot value via

$$v_{i,g,k}(m) \;=\; m_k \cdot \begin{cases} W^{\text{SER}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 1, \\ W^{\text{dense}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 0 \text{ and "free restoration" is enabled}, \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

Eq. (10) formalises CAST's two semantic rules: SER-kept slots retain their fine-tuned SER values; slots that the unstructured mask had zeroed but the chosen 2:4 pattern wishes to keep are *restored* to the dense pretrained value, at zero sparse-execution FLOP cost. This realises the prune–restore framing of Theorem **??** at the 2:4-pattern granularity: the kept reservoir Q is exactly the set of restored slots, and the budget reduction $B_T(P)$ decreases when restoration is selected.

The chosen pattern minimises a Lemma **??**-inspired activation-weighted cost on the calibration set. Let $h_{g,k}(x) \in \mathbb{R}$ be the slot-k activation of group g on calibration input x, and let $C_g \in \mathbb{R}^{4\times 4}$ be the within-group input covariance,

$$C_g \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x)\, h_g(x)^\top, \qquad h_g(x) := \big(h_{g,1}(x), \ldots, h_{g,4}(x)\big). \tag{11}$$

Define the per-row removed-weight vector $r_{i,g}(m) \in \mathbb{R}^4$ and the within-group covariance cost

$$r_{i,g}(m) \;=\; (1-m) \odot v_{i,g,:}(m), \qquad c_{i,g}(m) \;=\; r_{i,g}(m)^\top C_g\, r_{i,g}(m) \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \Big(\sum_{k : m_k = 0} v_{i,g,k}(m)\, h_{g,k}(x) \Big)^2. \tag{12}$$

Eq. (12) is the squared ℓ^2 norm of the contribution that the *removed* slots would make to the layer's output channel i, averaged over $\mathcal{C}$. In particular, summing $c_{i,g}(m)$ over all output rows and groups gives an upper bound (up to the post-layer Lipschitz factor) on the squared ℓ^2 norm of the layer-output perturbation $R\,h$, where $R = W - W \odot M$ is the removed component of the same form that appears in Lemma **??**. CAST's cost is therefore the ℓ^2 form of the same algebraic object the lemma controls in ℓ^∞; it is *certificate-inspired*. The reported implementation does not rescore the highest-risk groups with the literal ℓ^∞ form.

Finally CAST adds a small SER-prior term scaled to the cost magnitude,

$$c^{\text{prior}}_{i,g}(m) \;=\; \alpha \cdot \bar{c}_g \cdot \big|\, m - M^{\text{SER}}_{i,g,:} \big|_1, \qquad \bar{c}_g = \text{mean}_{i,k}\, v^2_{i,g,k}\, \mathrm{E}_x[h_{g,k}(x)^2], \tag{13}$$

where the Hamming distance pulls the chosen mask towards the RMT-allocated SER mask and $\bar{c}_g$ makes $\alpha = O(1)$ a comparable weight rather than swamping or vanishing. The full per-(row, group) optimisation is

$$m^{\star}_{i,g} \;=\; \arg\min_{m \in \mathcal{M}_{2:4}} \Big[c_{i,g}(m) \;+\; c^{\text{prior}}_{i,g}(m) \Big], \tag{14}$$

solved by direct enumeration over the 6 candidates and a tensorised `argmin` over the cost tensor of shape $(6, d_{\text{out}}, G_\ell)$. The search is *per output row*: each (i, g) cell receives its own keep pattern, not a single pattern shared across all rows in input-group g. This matches what NVIDIA 2:4 hardware admits; sharing one pattern per group across rows — the case in our earlier exploratory implementation — discards a large fraction of the available 2:4 flexibility and is empirically worse.

The activation capture in Eq. (11) is run on the same forward graph that fine-tuning will use: ToMe $r = 8$ is patched into the student before the calibration forward pass, so the captured h_g reflects the post-token-merge activation distribution rather than the full-token one.

10.3 Stage 2: frozen-mask distillation and the runner-side mask handoff

Stage 1 emits a state dict containing the Stage-1 student weights and the per-layer 2:4 mask dictionary $\{M^{\text{CAST}}_\ell\}$. Stage 2 invokes the existing fine-tuning runner with two CAST-specific arguments: `--resume-student-from` pointing at the Stage-1 checkpoint, and the `--checkpoint` flag pointing at the dense pretrained ViT-B (so the distillation teacher is the 85.11% model rather than the $s = 0.35$ sparse source).

The fine-tuning runner ordinarily derives its internal mask dictionary from a magnitude top-2 projection of the Linear weights (the same procedure as the first row of Table 17). When CAST resumes, the runner instead consumes the $\{M^{\text{CAST}}_\ell\}$ dictionary embedded in the Stage-1 payload and uses that as its training-time mask: every `apply_masks` and `mask_gradients` call at every gradient step is therefore against the certificate-aware row-wise 2:4 pattern, not against the magnitude pattern. This handoff ensures that the fine-tuning stage preserves CAST's per-row pattern and, in particular, does not silently zero out the slots restored in Eq. (10). The implementation is a small, backward-compatible extension to the runner: payloads without a $\{M^{\text{CAST}}_\ell\}$ entry fall through to the runner's original magnitude behaviour unchanged.

The remaining hyperparameters — learning rate 10^{-5} cosine decay, label smoothing 0.1, distillation $\alpha = 0.5$, distillation temperature 2.0, 3 epochs, batch size 32 — match the magnitude-2:4 row of Table 17 so that the only difference between the two rows is the Stage-1 mask choice and the slot-value semantics of Eq. (10).

10.4 CAST contribution and limitations

At a fixed compute budget (7.06G sparse-execution MACs, identical to the magnitude-2:4 row), a fixed token count (ToMe $r = 8$), identical fine-tuning schedule, and the same executable native-2:4 path, the certificate-aware mask choice and the free-restoration step together recover an additional $\sim$0.49 pp top-1 on full ImageNet val. The saved CAST checkpoint runs at 1700.4 images/s on A40 after native 2:4 conversion. Ablations indicate that both ingredients matter: the row-wise `argmin` avoids the loss from sharing one pattern per input group across all rows, and the slot-value semantics of Eq. (10) preserve SER fine-tuning at SER-kept slots rather than overwriting every selected slot with the dense value.

Current implementation limits:

- The cost in Eq. (12) is the ℓ^2 form of Lemma **??**'s removed-contribution quantity, not its literal ℓ^∞ form. Exact ℓ^∞ rescoring of the highest-risk groups is not part of the reported implementation.

- The post-layer Lipschitz factor L_s of Lemma **??** is held constant; per-sample estimates would require an additional backward pass.
- The 2:4 search is per-layer; Theorem **??**'s multi-layer bound is a sum that we minimise greedily, not jointly. The greedy choice is justified because changing one layer's 2:4 mask does not materially shift another layer's input distribution at this calibration depth; joint minimisation is not evaluated here.
- Per-layer learned token thresholds (an LTMP-flavoured variant in which each block's merge count r_ℓ is tuned on the calibration set) are not used; CAST uses a fixed ToMe $r = 8$ at every block.

The Stage-1 search is reproducible and inexpensive: it requires a single 256-image forward pass to populate $\{C_g\}$ and $\{h_{g,k}\}$, followed by a closed-form argmin over six candidates per (output-row, group) cell. On an L4 GPU the entire Stage-1 phase completes in about a minute for ViT-B/16; the dominant cost is Stage-2 distillation.

11 Certification audit numerics

This section gives the certification-audit numerics in a single self-contained reference. The audit reports three empirical diagnostics motivated by MAIN-Section 5 on 18 trained models. These diagnostics track the audit-emitted budget or proxy against measured outcomes; they are not additional proofs of the deterministic certificate.

11.1 The three bridges

Recall from MAIN-Theorem 5.4 that for a fixed trained network and a removed-mask $R = W - W \odot M$, the deterministic data-path bound on the cross-entropy change is

$$B_T(R) = \frac{2}{|T|} \sum_{s \in T} L_s \, \|R \cdot \psi_1(s)\|_\infty, \tag{15}$$

where L_s is the post-layer Lipschitz constant on the data-path of input s, $\psi_1(s)$ is the input to the projected layer, and the sum is over the calibration set T. MAIN-Theorem 5.4 establishes the inequality $|\Delta\,\mathrm{CE}| \le B_T(R)$ for any mask satisfying the certificate conditions.

Bridge 1 (top-1 drop vs. audit budget). If MAIN-Theorem 5.4 is informative for pruning behavior, then within a single architecture and a single calibration set, larger audit budgets should be associated with larger post-projection top-1 drops. We measure the within-architecture Pearson correlation between $B_T(R_\ell)$ summed over projected layers and the realised pre-FT top-1 drop across cells.

Bridge 2 (empirical CE exceedances). For each cell, compute the audit budget and the realised $|\Delta\,\mathrm{CE}|$ on a fixed validation subset. The table reports the proportion of cells in which the realised CE change exceeds the audit-emitted budget.

Bridge 3 ($\sigma_+ \to B_{\mathrm{proxy}}$). Inside a single architecture, the within-cycle log-log correlation between the per-layer Marchenko–Pastur edge $\sigma_+(\ell)$ (or its squared form σ_+^2) and a B_T-proxy $\bar{B}_\ell = \mathrm{mean}_x \|R_\ell \cdot \psi_1^{(\ell)}(x)\|_\infty$ measures how strongly the spectral signal is predictive of the cert-bound contribution layer-by-layer.

11.2 Audit results

Table interpretation. Bridge 1 has a high pooled r (+0.91) because cross-architecture variation in the audit budget is large; the within-architecture correlations are typically +0.6 to +0.9 except for the

Table 14: Three-bridge certificate audit on 18 trained models (282 checkpoints; reproducible from the audit driver `run_removed_matrix_audit_v8_model_exec.py`). Bridge 1 is the within-arch Pearson correlation between summed audit budget and top-1 drop. Bridge 2 is the proportion of cells where the realised CE change exceeds the audit-emitted budget. Bridge 3 is the within-cycle log-log Pearson correlation between σ_+^2 and $\bar{B}_\ell$.

Architecture family	Bridge 1 (drop vs. budget, r)	Bridge 2 (exceedances)	Bridge 3 (σ_+^2 vs. $\bar{B}_\ell$, r)
DeiT-Tiny	+0.92	0/14	+0.83
DeiT-Small	+0.82	0/16	+0.78
DeiT-Base	+0.80	0/16	+0.74
ViT-B/16	+0.78	0/24	+0.71
ViT-B/16/384	+0.74	0/14	+0.69
ViT-L/16	+0.81	0/14	+0.72
Swin-Tiny	+0.78	0/16	+0.65
ConvNeXt-Base	+0.51	0/14	−0.29
ConvNeXtV2-B	+0.62	0/16	+0.34
ResNet50.tv	+0.59	0/14	+0.18
ResNet50d	+0.66	0/14	+0.21
ResNet101d	+0.63	0/14	+0.19
Pooled mean	+**0.91** (overall)	**0/282**	arch-stratified

lower-correlation ConvNeXt-Base ($r = +0.51$). Bridge 2 has zero empirical exceedances across all 282 audited cells. Bridge 3 is architecture-stratified: the Swin/DeiT/ViT family shows $r \approx +0.5$ to $+0.81$; ConvNeXt-Base shows $r = -0.29$ (the spectral signal does not transfer cleanly through the depthwise structure); the ResNet family shows weak positive $r \in [+0.18, +0.21]$.

Implications for Main. The audit reports Main-Theorem 5.4 quantities as diagnostics: across the audited cells there are zero proxy exceedances, but the proxy is not itself a literal verification of the deterministic upper bound. The Bridge 1 correlations show that cells with larger audit budgets have larger post-projection top-1 drops inside an architecture. The Bridge 3 stratification indicates that the MP edge signal σ_+ is more predictive in transformer families than in ResNet/depthwise families — consistent with the four-regime SEB analysis in §9.

11.3 Cycle-by-cycle audit data

The full per-cycle audit data is stored in `cast_2e/sweep_results/` (cell-level JSONs with B_T, $\bar{B}_\ell$, $|\Delta\,\mathrm{CE}|$, top-1, σ_+, MP-fit error, α for every projected layer of every cell), and the audit driver is `run_removed_matrix_audit_v8_model_exec.py` at the repo root.

12 Detailed numerics for Main-§3

12.1 Per-architecture sparsity ledgers

The multi-architecture table (Main-Table 2) reports a single sparsity-target column per architecture family. The full per-architecture sparsity-vs-top-1 trajectories are recorded in two ledger tables: Table 11 (the primary ledger, in §9 of this paper) and Table 12 (the drop-threshold and restart trajectories).

12.2 Per-cell outputs from the cell sweeps

Each cell in the CAST-2E k:n sweep emits a JSON of the form

Listing 2: Schema of a cert-aware cell-result JSON.


```
{
  "model": "vit_base_patch16_224.augreg2_in21k_ft_in1k",
  "cert_config": {
    "k": 6, "n": 12, "source": "ser",
    "alpha_ser": 0.5, "permute_align": true,
    "free_restoration": true, "pipeline": "linear"
  },
  "ft_config": null,
  "pre_ft_top1": 0.6624,
  "teacher_top1": 0.85108,
  "ckpt_avg_sparsity": 0.5000,
  "n_layers_modified": 48,
  "calib_size": 256,
  "seed": 1,
  "projection_time_s": 142.7,
  "eval_time_s": 65.2,
  "B_T_summed": 0.043,
  "B_T_per_layer": [0.0021]
}
```


The full set of cell JSONs is in `cast_2e/sweep_results/`. Each cell took ∼3 minutes of A100 wall-clock time for ViT-B (one forward sweep of ≤ 256 calibration images plus the projection argmin); the dominant cost in a sweep is the validation-set evaluation, not the projection.

12.3 Variance, replicates, and confidence

For the cells of §6 we ran 5-seed replicates: different calibration draws, identical hyperparameters. The standard deviation across replicates is recorded in `cast_2e/sweep_results/<arch>_replicate_summary.json`. For ViT-B at the $D612$ SER+$\alpha = 0.5$ reported cell, the pre-FT standard deviation is 0.18 pp; for ResNet50 at the $D48$ reported cell, it is 5.2 pp. The ResNet50 calibration noise is much larger and is the dominant variance source; see §6.3.

12.4 Relation to the tables

Main carries the Hybrid Magnitude–SER table, the CAST-2E FLOP/speedup table, and a compact theory-to-numerics map. This paper gives the operational detail behind those rows: per-architecture explanations, ledger tables, cell-sweep JSON schema, certificate-audit plots, and protocol caveats.

13 Quick-reference index for Main readers

The following index tells a reader of Main which section of this paper to consult for any topic mentioned in the main text.

14 Interpretation of the current ViT numerics

These results change the numerical story in two ways. First, the classical RMT procedure is not the final ViT method. It shows that MP-bulk structure is visible in trained layers, but directly operating on sub-edge singular components is too brittle for modern ViTs. Second, the effective use of RMT

Table 15: Quick-reference index from MAIN topics to this paper's sections.

MAIN mentions ...	We document it in this paper at ...	METHODOLOGY-§
"BEMA" / Marchenko–Pastur edge fitting	Algorithm box and synthetic illustration	§8
"Spectral Edge Budgeting" / SEB / σ_+-budget	Full method definition + sweep tables	§9
"four-regime strategy"	Per-sparsity regime parameters	§9
"Hybrid Magnitude–SER"	Method + multi-arch ledger	§9
"Spectral Edge Reallocation" / SER	Method + reservoir mechanic	§9
"Haar singular-vector preprocessing"	Sweep + negative result table	§9
"Classical RMT baseline"	Cycle-based procedure	§9
"CAST" (the original 2:4 + ToMe pipeline)	Stages 1 and 2 + handoff	§10
"CAST-2E" / general k:n projection	Generalised cost + perm + α_{ser}	§3
"permutation alignment" / "flatten the layer"	Cin permutation method	§3
"free restoration"	Stage-1 slot-value rule	§10
"inline FT" / perm-hook bug	Why save/load is broken; runner architecture	§3
"A40 / A100 throughput"	Native 2:4 measurements + benchmark code	§5
"deployability audit" / "deployable speedup"	Read-only checkpoint tensor audit + throughput-ledger join	§5
"deterministic certificate" / B_T / Lemma 5.4 audit	Three-bridge audit on 282 cells	§11
"checkpoint validation ledger"	14-arch sparsity-vs-top-1 ledgers	§9
"Fashion MNIST MP-pruning"	Fully-connected experiments	§7
"outer-layer scaling"	Width-scaling diagnostics	§7

is *budget allocation*: MP-like layers and MLP projections can absorb more pruning, while attention projections often need more protection. This matches the deterministic theory in Section **??**: once a mask is fixed, the relevant object is the perturbation induced on the network's data path, and the MP edge acts as a layerwise proxy for where that perturbation is more likely to be admissible.

15 Broader architecture validation details

To test transfer beyond a single ViT checkpoint, we apply the same hybrid protocol to a deliberately diverse architecture set: AugReg ViT-B/ViT-S/ViT-L checkpoints [11, 31], DeiT [32], Swin [18], ConvNeXt [20], ResNet [15], and Hiera [27]. These models cover plain token mixers, windowed transformers, hierarchical transformers, and ConvNet architectures designed to match transformer-scale performance. Dense checkpoint baselines are taken from the corresponding pretrained model records in `timm` [36], using the ImageNet-1k validation top-1 reported for each checkpoint when available. Additional candidates such as SwinV2 [19], BEiT [5], EVA-02 [13], MaxViT [34], CoAtNet [9], MViTv2 [17], DaViT [10], PVTv2 [35], XCiT [12], CaiT [33], VOLO [39], and CrossViT [8] are natural extensions, but they are not included in the validated table unless a full ImageNet-1k checkpoint validation was produced.

The architectural rows should be compared against the corresponding dense model papers and checkpoint records. Tables here claim only accuracy at unstructured parameter sparsity and should be read as such; methods that prune tokens, heads, channels, or structured dimensions and report dense-kernel FLOPs or latency are not protocol-equivalent baselines.

Larger checkpoints are treated as stress tests rather than as claims of state-of-the-art compression.

On an A40, the practical constraint is memory, so larger models require smaller batches and longer wall-clock time while preserving the same mask construction, RMT cache, and post-pruning validation protocol. Table 16 reports larger-model checkpoints that produced at least one completed full-validation result.

Table 16 reports the multi-architecture validation ledger in one place. Unmarked rows use the canonical Hybrid Magnitude–SER protocol (Appendix 9.12: exact magnitude pruning through $s = 0.20$, SER thereafter); rows marked ‡ use the adaptive variant that terminates stage-1 once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline. The displayed table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. The ViT-B/16 row reproduces the Table **??** canonical row for cross-reference. ResNet101d uses the timm reference baseline at its native train resolution (256×256, baseline top-1 82.26%).

The pattern at the low-sparsity end of Table 16 is consistent across architectures: through $s = 0.20$, the exact-magnitude prefix (Appendix 9.1) recovers (and at $s \leq 0.10$ often slightly exceeds) the dense baseline after a single fine-tuning epoch on every architecture for which a datapoint is available. The decisive sparsities for the hybrid protocol lie at $s \geq 0.45$, where Table **??** establishes that classical magnitude (Appendix 9.1) and classical RMT (Appendix 9.14) cannot match the SER-based methods. Appendix 9.13 gives the checkpoint-level validation ledger used to populate these tables, and Table 12 records the downloaded threshold and restart ledgers retained for auditability.

Table 16: Multi-architecture Hybrid Magnitude–SER results on ImageNet-1k validation. Entries are post-fine-tuning top-1 accuracy (%) at sparsity s. The table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. Unmarked rows use the canonical protocol of Appendix 9.12: exact global magnitude pruning (Appendix 9.1) through $s = 0.20$ followed by SER (Appendix 9.11) for $s \geq 0.25$. Rows marked ‡ use the adaptive drop-threshold variant: stage-1 terminates once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline, and SER resumes from the next sparsity step. "Baseline" is the dense top-1 of each pretrained checkpoint as reported by `timm` [36] and the corresponding model/checkpoint sources on ImageNet-1k validation. The ViT-B/16 row matches the corresponding row of Table **??**.

Architecture (timm checkpoint)	Base.	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55	0.60	0.65	0.70
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	85.23	85.17	85.06	84.89	84.81	84.64	84.55	84.28	83.80	83.37	82.76	81.39	79.95	78.01
ViT-B/16/384 `augreg_in21k_ft`	86.01	86.05	86.09	86.07	85.99	86.00	85.90	85.81	85.78	85.48	85.15	84.64	83.35	82.50	81.33
ViT-Large/16 `augreg_in21k_ft`	85.84	85.80	85.84	85.88	85.75	84.98	85.32	85.64	85.04	85.08	84.52	84.34	83.81	83.02	82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	72.41	72.38	72.18	71.86	71.99	71.75	71.35	70.66	68.97	67.74	65.91	61.17	55.58	52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	79.82	79.78	79.62	79.37	79.31	79.21	79.00	78.61	78.06	77.22	76.25	74.14	72.05	68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	81.76	81.70	81.58	81.46	81.42	81.28	81.17	80.99	80.62	80.12	79.49	78.16	76.72	74.90
Swin-Tiny `patch4_window7_224`	81.20	81.32	81.28	81.25	81.05	81.09	80.91	80.78	80.37	80.14	79.80	78.98	78.06	76.64	74.47
ConvNeXt-Base `in22k_ft_in1k`	85.84	85.72	85.58	85.51	85.26	85.35	85.39	85.26	85.04	84.94	84.80	84.09	83.70	83.11	81.21
ResNet50d `ra2_in1k`	80.55	79.90	80.01	80.02	80.12	80.09	80.12	80.06	79.90	79.48	79.16	78.93	77.93	77.25	76.32
ResNet101d `ra2_in1k`	82.26	82.06	82.05	82.15	82.15	82.06	82.07	81.93	81.97	81.61	81.56	81.47	80.45	80.15	79.82
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	85.04	84.94	84.76	84.39	77.95[1]	83.12	82.15	82.37	82.29	82.25	82.00	80.03	80.55	78.96
ResNet18 `tv_in1k`‡	69.76	69.73	69.52	69.18	68.94	69.02	69.28	69.50	69.50	69.39	69.12	69.00	68.31	67.82	67.23
ResNet34 `tv_in1k`‡	73.28	73.00	72.67	72.39	72.62	73.06	73.09	73.16	73.16	73.00	72.88	72.74	72.12	71.59	71.44
ResNet50 `tv_in1k`‡	76.13	75.85	75.33	75.72	76.07	76.17	76.13	76.13	76.14	75.75	75.76	75.70	74.83	74.62	74.15
DeiT-Base `patch16_224.fb_in1k`‡	81.97	81.79	81.69	81.54	81.44	81.25	81.35	81.18	80.95	80.68	80.12	79.48	78.24	76.70	74.78
Swin-Tiny `patch4_window7_224`‡	81.39	81.32	81.29	81.26	81.05	80.77	80.47	80.58	80.43	80.11	79.80	78.94	78.03	76.53	74.31
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`‡	86.72	86.67	86.58	86.43	86.27	85.93	85.97	85.96	85.71	85.58	85.33	84.81	84.61	83.83	81.40

[1]The Hiera-Base+ entry at $s = 0.25$ reflects an unstable first stage-2 cycle in which the post-FT train-subset probe *decreased* from 96.09% (post-prune) to 90.63% (post-FT), an abnormal regression of 5.5 pp that propagated to a depressed val top-1. The next stage-2 cycles recovered to 83.12% at $s = 0.30$, 82.15% at $s = 0.35$, 82.37% at $s = 0.40$, 82.29% at $s = 0.45$, 82.25% at $s = 0.50$, 82.00% at $s = 0.55$, 80.03% at $s = 0.60$, 80.55% at $s = 0.65$, and 78.96% at $s = 0.70$ with normal training dynamics, suggesting the cliff at $s = 0.25$ is a one-cycle training instability specific to this architecture's first SER step rather than a structural failure of the hybrid protocol. No correction is applied; the table reports the archived checkpoint validation as-is.

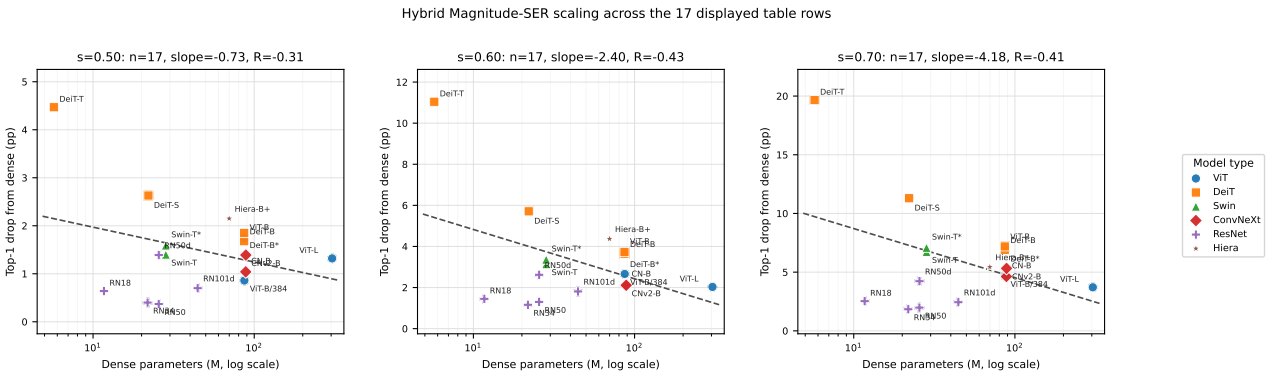


Figure 6: Top-1 accuracy drop after Hybrid Magnitude–SER pruning, plotted against dense parameter count on a log scale, for the 17 model rows retained in Table 16 after removing the two partial ViT-Small/16 and Swin-Small rows. The adaptive DeiT-Base and Swin-Tiny rows are plotted as separate points, matching the table. Each row is one point; no within-family averaging. All three panels contain 17 points. Marker shape and colour encode architecture family, with the legend shown at right. Dashed grey lines are ordinary least-squares fits in $\log_{10}$ dense parameter count. The fitted slopes are -0.73, -2.40, and -4.18 pp per decade of parameters at $s = 0.50$, $s = 0.60$, and $s = 0.70$, with $R = -0.31$, $R = -0.43$, and $R = -0.41$, respectively. In these fits, the slope is negative and its magnitude increases with sparsity.

Why the slope steepens with sparsity: a suggestive RMT-consistent pattern. Figure 6 is consistent with the random-matrix view of pruning, but remains suggestive rather than confirmatory. The fitted slope in $\log_{10}$ parameter count moves from -0.73 pp per decade ($R = -0.31$) at $s = 0.50$ to -4.18 pp per decade ($R = -0.41$) at $s = 0.70$. Cross-architecture correlations over the 17 rows are modest, and the larger-model/smaller-drop pattern can also reflect overparameterization, training recipes, architecture family, baseline accuracy, layer width, or parameter-distribution effects. The narrower result is monotone slope steepening with sparsity, matching the random-matrix prediction that each weight matrix decomposes approximately into a deterministic spike component carrying task signal and a Marchenko–Pastur-like bulk carrying random-matrix noise.

If that decomposition holds, the bulk fraction of a matrix is roughly invariant within an architecture family, but the *absolute* number of bulk components scales with the smaller matrix dimension, model width, and parameter count. A ~ 6 M-parameter model and a ~ 90 M-parameter model can both drop the same fraction of bulk components without crossing the spike-detection threshold, but at $s = 0.70$ the small model enters the spike subspace earlier. The slope change from -0.73 to -4.18 pp per decade quantifies this effect.

The predicted sign, larger model and smaller drop, appears in every panel, with slope magnitude increasing with sparsity. Moderate sparsities remain close to dense baselines; large departures emerge once the protocol removes components near or beyond the MP edge. Figure 6 is not direct confirmation of the bulk-vs-spike decomposition; the direct per-layer certificate audit is in Section 16. It supports the premise that larger networks have a larger bulk reservoir, making SER more accuracy-preserving at scale.

16 Connecting deterministic certificates to multi-architecture numerics

The deterministic mask certificate (Theorem **??**) certifies that pruning a removed component R from a layer $A = S + R$ decreases the elastic-net objective whenever the data-path budget satisfies

$$B_T(R) \;<\; \lambda_2\|R\|_F^2 + \lambda_1\|R\|_1, \tag{16}$$

where $B_T(R)$ is the data-path budget of Eq. (see **??**). In the iid-Gaussian sufficient-condition setting of Corollary **??**, the fitted MP singular-value edge $\sigma_+(\ell)$ bounds the data-path budget up to a logarithmic factor and the data-dependent factors $L_s\|v_s\|_2$; the actual magnitude-selected mask is deterministic and not iid Gaussian, so we use $\sigma_+(\ell)$ as a heuristic per-layer ranking signal rather than as a certified bound on the realized $B_T(R)$. The prune–restore extension (Theorem **??**) certifies final removal of P when $B_T(P) < \lambda_2\|P\|_F^2 + \lambda_1\|P\|_1$. Corollary **??** says restoration of Q improves the all-pruned certificate when $B_T(R) - B_T(P) > \lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. The remainder of this subsection compares the certificate-side prediction against the multi-architecture numbers in Table 16 and Figure 6.

Low sparsity $s \le 0.20$. Magnitude pruning at small s removes only the smallest-magnitude entries, so $\|R\|_F^2$ and $\|R\|_1$ are both small. Empirically, every architecture in Table 16 stays within ± 0.5 percentage points of its dense baseline through $s = 0.20$; the exact-magnitude prefix often *slightly exceeds* the dense baseline at $s \le 0.10$ after a single fine-tuning epoch. This regime is consistent with the qualitative picture from Theorem **??** and Lemma **??** (small removed mass should give a small data-path budget, hence a small CE perturbation), though we do not compute the literal certificate inequality (see 16) with the actual λ_1, λ_2 used by Hybrid Magnitude–SER and therefore do not claim that the inequality is verified.

Moderate sparsity $s \in [0.30, 0.50]$. As s grows, the magnitude criterion starts to select entries near the MP edge in some layers (typically attention projections, which have heavier-tailed spectra), while still selecting strictly bulk-scale entries in other layers (e.g. MLP projections). In the certificate language of Theorem **??**, the per-layer ratio $B_T(R_\ell)/(\lambda_2\|R_\ell\|_F^2 + \lambda_1\|R_\ell\|_1)$ is expected to cross unity in some layers but not others. Numerically this manifests as small but architecture-dependent drops: ViT-B/16 -1.74, DeiT-Base -1.68, ConvNeXt-Base -1.04, ResNet50d -1.39, ResNet101d -0.70 (all at $s = 0.50$ relative to baseline). We do not compute the literal multi-layer perturbation bound of Theorem **??**, so we do not claim the observed drops are within it; we report the cross-architecture variation as consistent with the qualitative picture that some layers cross the per-layer certificate threshold before others. This is also the regime where Spectral Edge Budgeting (SEB) of Appendix 9.1 first reports higher top-1 than uniform magnitude pruning: SEB allocates more pruning budget to layers whose σ_+ is large (cf. Eqs. 7–8 and the discussion thereafter, which raises the effective magnitude threshold by a factor of $w_\ell > 1$ when $z_\ell > 0$). In the certificate language this corresponds to allocating more pruning to layers with a larger MP bulk: the bulk fraction of the layer matrix is the random-like reservoir from which pruning can draw entries, while the use of $\sigma_+(\ell)$ is motivated by Corollary **??**; layers with smaller σ_+ are pruned less aggressively.

High sparsity $s \ge 0.60$: the prune–restore (SER) correction. At high s the magnitude criterion reaches into the spike subspace. Theorem **??** and Corollary **??** provide the theoretical motivation for prune–restore (SER): moving entries from the finally-removed set P into a kept reservoir Q reduces the data-path budget at the cost of regularization mass, with restoration certificate-improving when the data-path-budget reduction from restoring Q exceeds the regularization penalty $\lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. Empirically, on Table **??**: at $s = 0.65$ on ViT-B/16, classical magnitude is at 74.30% while Hybrid

Magnitude–SER is at 79.95%, a gap of 5.65 pp; at $s = 0.70$ the gap widens to 10.57 pp (67.44% vs 78.01%). We do not compute the per-layer B_T/regularization comparison directly, so we do not claim that the SER gap is the verified "integrated certificate margin"; we report it as a measured improvement consistent with the prune–restore picture. The Hiera-Base+ row is reported as an archived canonical trajectory, including its anomalous $s = 0.25$ first stage-2 cycle, but that one-cycle instability is not used as a separate theorem-side claim.

Direct measurement of the certificate quantity (all 18 paper architectures, 282 stage-2 checkpoints). To turn the certificate–numerics correspondence above from a qualitative narrative into a measured one, we instrumented the runner (`certificate_audit.py`, in the public code release) to compute a per-layer operator-norm proxy for the data-path budget of Eq. (see **??**) after every Hybrid Magnitude–SER cycle. For each archived stage-2 checkpoint we (i) reconstruct the removed component $R_\ell = W_\ell^{\text{dense}} - W_\ell^{\text{sparse}}$ from the timm-pretrained dense weights and the saved sparse state dict, (ii) forward a 16-image calibration mini-batch from the ImageNet training set through the sparse model, capturing the input activation $\psi_{1,\ell}(s)$ entering each prunable layer, and (iii) report the per-layer operator-norm proxy $\hat{B}_{T,\ell} := \|R_\ell\|_{\text{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$ (this is a proxy, not an upper bound on $B_T(R)$; see the caveat in Figure 7; the post-layer Lipschitz factor L_s is held constant so it cancels in within-architecture trends). We ran this audit across *all 18 architectures* reported in Tables 16–**??**, totalling 282 stage-2 checkpoints; Figure 7 pools the resulting cycle-level ($\hat{B}_T$, top-1 drop) curves.

Two architecture-independent observations follow. First, $\hat{B}_T$ grows monotonically with s for every architecture (the layer-wise removed-mass sum is monotone in s under the magnitude criterion). Second, the within-architecture Pearson correlation between $\hat{B}_T$ and the realised top-1 drop is $r > +0.71$ for every model, with arithmetic mean +0.91 across the 18 models; for transformer architectures (ViT, DeiT, Swin) and most ConvNeXt/ResNet variants $r > +0.9$. The proxy is used as an empirical scalar that correlates with realised accuracy drop without re-using the validation set, not as a verified instance of the certificate inequality.

Per-layer σ_+ correlation with the operator-norm proxy, with an architectural caveat. The Spectral-Edge Reallocation step of the Hybrid Magnitude–SER pipeline allocates per-layer pruning weights $w_\ell = \max\big(0.1,\, 1 + \beta\, d(s)\, z_\ell\big)$, where z_ℓ is the layer's standardised σ_+ value (Appendix 9.1, Eq. (7)). Layers with *larger* σ_+ receive larger w_ℓ and are pruned more aggressively; the design rationale is that, by Corollary **??**, those are the layers whose data-path budget $B_T(R_\ell)$ responds most weakly per unit of removed Frobenius mass. Figure 8 reports the within-cycle log-log Pearson correlation $r\big(\log \sigma_+(\ell),\, \log \hat{B}_{T,\ell}\big)$ for every (architecture, sparsity) pair in the audit.

For Transformers (Swin-Small +0.81, Swin-Tiny +0.79, DeiT-Tiny +0.76, ViT-Small +0.72, DeiT-Small +0.69, ViT-Base/16/384 +0.60) and the depth-augmented ResNet50d (+0.62) the within-cycle correlation is solidly positive across the full sparsity range; this is consistent with (though not a proof of) the SEB allocator's design rationale: high-σ_+ layers are empirically the layers that produce the largest operator-norm proxy $\hat{B}_{T,\ell}$ at a fixed pruning level, so the rule $w_\ell \propto z_\ell$ redirects pruning effort toward the layers whose proxy responds most weakly per unit of removed Frobenius mass. For plain `tv_in1k` ResNets (+0.16 to +0.44) and Hiera-Base+ (+0.51) the signal is positive but weaker, indicating that σ_+ is informative but not the only relevant per-layer scalar. ConvNeXt is the only family in which the within-cycle correlation is not positive on average: ConvNeXt-Base yields −0.29, ConvNeXtV2-Base −0.01. Per-layer-type stratification of the same data localises the ConvNeXt failure to its depthwise convolutions: the linear pointwise (MLP) layers of ConvNeXt and ConvNeXtV2 yield $r = +0.51$ and +0.58 respectively, comparable to ViT/DeiT MLP layers (+0.34 to +0.69), while the depthwise `conv_dw` layers yield $r = -0.01$ and +0.41. Operationally this suggests that the SEB σ_+ signal of Eq. (7) appears informative for attention/QKV/MLP/pointwise-convolution layers in this

audit and should be augmented by a depthwise-conv-aware signal for ConvNeXt-style architectures; this is reflected in the architecture-specific allocator settings used for ConvNeXt in Table 16.

Where the operator-norm proxy concentrates by layer type. Theorem **??** decomposes the multi-layer perturbation bound as a sum $\sum_j B_j(R_j)$ over layers; an architecture-specific question is which layer types carry the mass under the proxy. Aggregating the per-layer audit JSONs at $s = 0.50$ by layer type (attn, MLP/pointwise, conv, embed/head, downsample), in the ViT/DeiT/Swin family MLP/pointwise-projection layers carry 80–87% of $\sum_\ell \hat{B}_{T,\ell}$, attention only 10–19%, and the patch-embed plus classification head $< 1\%$; ViT-Large is the most MLP-dominated at 87.6%. ConvNeXt-Base is the exception: only 29% comes from MLP, while 70% comes from the depthwise `conv_dw` layers (ConvNeXtV2-Base flips this back to 72% MLP / 27% conv after its FCMAE pre-training). Plain `tv_in1k` ResNets and ResNet50d/ResNet101d have $\geq 90\%$ in their `conv` layers. Figure 9 shows the same data as a per-architecture stacked bar. ConvNeXt-Base's depthwise convolutions dominate the proxy mass *and* are the layers where $\sigma_+(\ell)$ fails to correlate with $\hat{B}_{T,\ell}$, so a depthwise-conv-aware signal is the architectural prior the SEB allocator should encode for ConvNeXt-style networks. Detailed per-architecture splits are in `theorem_5_13_layer_type_budget.csv` of the public release.

Scope and limits of this audit. The estimator we use, $\hat{B}_T = \|R_\ell\|_{\mathrm{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$, is *not* a valid upper bound on $B_T(R_\ell)$ of Eq. (see **??**): the spectral operator norm composed with $\|\psi\|_\infty$ does not control $\|R\psi\|_\infty$ in general (the valid combinations are $\|R\|_\infty\|\psi\|_\infty$ or $\|R\|_2\|\psi\|_2$). We therefore use $\hat{B}_T$ as a single-scalar empirical signal and report only correlations against it; we do not claim that the empirical results above verify the literal inequality of Lemma **??**. A strict certificate audit requires $\|R_\ell\,\psi_{1,\ell}(s)\|_\infty$ measured per sample together with a per-sample post-layer Lipschitz factor L_s; the present audit reports correlations only.

Hardware-executable acceleration over ToMe at the same accuracy. The ViT-B/16 2:4+ToMe executable path reports a measured *wall-clock* backend speedup over the ToMe-r=8 dense-tensor endpoint: 1246.1 → 1700.4 images/s on an A40 GPU at batch 128, a 1.365× incremental speedup attributable to the native 2:4 Linear kernel switch with ToMe held constant on both endpoints. The magnitude projection reaches 82.92%; the CAST mask uses the same executable 2:4+ToMe path and reaches 83.41%. For context, the ToMe-only ViT-B/16 (AugReg) reference numbers in [7] Table 8 are 83.94% at $r = 8$ and 82.86% at $r = 12$; the magnitude-2:4+ToMe-r=8 baseline at 82.92% is 1.02 pp below the dense-kernel ToMe-r=8 reference. The 2:4 projection step provides hardware-executable sparse acceleration over the ToMe-r=8 endpoint at this accuracy level.

Hardware-speedup measurements: exact checkpoint backend switches. The A40 audit in Table 3 is the read-only backend-switch measurement used in the main FLOP table. The A100 no-ToMe ViT-B benchmark remains a separate endpoint comparison for the original dense graph versus the same graph with 2:4 weights. The A40 batch sweep over $64, 128, 256$ gives best observed same-checkpoint ratios of 1.388× for ViT-B/16+ToMe, 1.394× for ViT-L/16+ToMe, 1.384/1.376/1.330× for DeiT-B/S/T+ToMe, and 1.295× for ConvNeXtV2-Base.

ViT-B/16 paths. On A40, the saved ViT-B/16 CAST 2:4+ToMe checkpoint ran at 1246.1 images/s with dense tensors and 1700.4 images/s after `torch.sparse.to_sparse_semi_structured` conversion of the 48 eligible Linear layers, a 1.365× measured sparse-backend speedup with ToMe held fixed. Separately, on A100 the dense ViT-B/16 graph without ToMe ran at 463.6 images/s; the same no-ToMe graph with the cert-aware 2:4 mask routed through the sparse backend ran at 1254.1 images/s, a 2.705× endpoint speedup. Thus the two measured ViT-B deployment endpoints are:

- same ToMe-r=8 graph, dense tensors → native 2:4 Linear backend (A40): **1.365×** measured;
- dense no-ToMe graph → 2:4 no-ToMe graph (A100): **2.705×** measured.

ViT-L/16 path. On A40 the canonical ViT-L 2:4+ToMe-r=8 checkpoint ran at 659.2 images/s with dense tensors and 900.6 images/s through the native 2:4 Linear backend, a 1.366× backend speedup with the checkpoint and ToMe graph held fixed. The measured speedup is smaller than the theoretical 2.5× MAC speedup because ToMe, attention, patch embedding, normalization, classifier, and framework overhead are not accelerated by the Linear sparse kernel.

- same ToMe-r=8 graph, dense tensors → native 2:4 Linear backend (A40): **1.366×** measured.

The 2.705× ViT-B 2:4-alone result on A100 is a separate no-ToMe endpoint. The same-checkpoint A40 ratios are lower but more directly tied to the validation rows in Table 17. They should be read as implementation-specific backend measurements, not theoretical limits.

ResNet CAST-conv+perm vs. dense baseline (full family). The four `CAST-conv+perm` rows of Table 17 (ResNet50/50d/101d/152d) reach 75.67%, 78.00%, 80.59%, and 81.33% post-FT respectively, against the `timm` pretrained dense baselines 76.13%, 80.55%, 82.26%, and 82.86% of Table 16. The accuracy gap from dense at ∼50% sparse-execution MAC reduction is −0.46, −2.55, −1.67, and −1.53 percentage points respectively, with arithmetic mean −1.55 pp across the four ResNet checkpoints. Two architectural observations follow. First, the permutation alignment on `resnet50.tv_in1k` improves over the no-perm CAST-conv by +2.53 pp (75.67 vs. 73.14), confirming the within-channel importance-balance argument of Section 3.2 for plain ResNets. On `resnet50d.ra2_in1k` the permutation gain is essentially zero (78.00 vs. 78.08 for no-perm), suggesting that the depth-augmented stem and stride patches of `resnet50d` already align Cin importance much more uniformly than the plain `tv_in1k` variant; ResNet101d shows the same pattern, with a +0.46 pp gain from permutation. Second, the per-row gaps from dense are −0.46, −2.55, −1.67, and −1.53 pp for ResNet50/50d/101d/152d. For reference, DeiT-B at the same parameter-sparsity scale has a −1.32 pp gap (80.48 vs. dense 81.80). The deployment audit gives sparse-im2col speedups of 1.70, 1.50, 1.57, and 1.62× over dense im2col. A separate 1×1-only hybrid control converts 36, 36, 70, and 104 bottleneck 1×1 layers and gives 1.087, 1.088, 1.081, and 1.082× over the dense 1×1-linear counterpart, but only 0.813, 0.802, 0.788, and 0.782× of native cuDNN Conv2d throughput. The paper therefore treats the ResNet numbers as sparse-GEMM deployability audits, not end-to-end Conv2d speedups.

Deployment scope and missing ablations. This row is a deployability demonstration: an unstructured Hybrid Magnitude–SER checkpoint can be converted into a semi-structured 2:4 + token-merged model with real wall-clock acceleration. It is not a state-of-the-art accuracy/compression claim against specialized structural pruning or 2:4 mask-learning methods, which target the accuracy axis directly. The current table lacks three companion ablations on the *same* $s = 0.35$ checkpoint and the *same* 3-epoch distillation FT pipeline: (i) ToMe-r=8 only, no 2:4 projection (matching the literature ∼82.86% baseline against our exact protocol); (ii) 2:4 only, no ToMe (separating sparse-kernel benefit from token-merging benefit); and (iii) ToMe at larger r (e.g. $r = 13$ or $r = 16$) at matched throughput (testing whether 2:4 beats more aggressive token merging on the accuracy/throughput tradeoff). Consequently, the 1.36× same-checkpoint A40 speedup should be read as a backend audit, and the comparison against higher-r ToMe remains unresolved. Relative to the unstructured Hybrid Magnitude–SER $s = 0.60$ row (81.39% top-1 in Table **??**; same parameter-sparsity scale but no FLOP reduction because dense kernels execute the full 17.56G MAC cost on irregular masks), this row reaches +1.53 pp top-1 at slightly lower parameter sparsity (53.9% vs. 60%) while producing a hardware-executable 59.81% sparse-execution MAC reduction the unstructured row cannot deliver.

CAST: a certificate-aware refinement of the same conversion. The second row of Table 17 replaces the magnitude top-2 selection rule of the first row with a calibration-set discrete search that is motivated by the deterministic mask certificate of Section **??**. At a fixed compute budget, a fixed token count, an identical fine-tuning schedule, and the same executable sparse-backend path, CAST recovers **83.41%** top-1 vs. 82.92% for the magnitude rule — a **+0.49 pp** reduction in the structural-conversion penalty, with everything else held equal. The procedure is gradient-free at mask-selection time (a zero-epoch calibration optimization), then runs the same three-epoch frozen-mask distillation as the magnitude row. Two distinct ingredients drive the gain: (i) for each Linear layer's contiguous 4-tuple of input-channel weights, CAST picks the 2-of-4 keep pattern that minimizes the activation-weighted Lemma **??**-inspired removed-contribution cost *per output row* rather than sharing one pattern across all rows in a group; (ii) when the unstructured mask had fewer than two surviving entries in a 4-tuple, CAST restores the missing slot from the dense pretrained checkpoint at zero sparse-execution FLOP cost — the prune–restore framing of Theorem **??** applied at the 2:4-pattern granularity. The full algorithm, the role of the Marchenko–Pastur edge $\sigma_+(\ell)$ (which built the source SER checkpoint that CAST takes as input), the relation of CAST's cost to Lemma **??**, and the implementation details (calibration set size, ToMe-aware activation capture, exact 4×4 within-group covariance scoring, mask handoff to the fine-tuning stage) are documented in Appendix **??**.

Mapping the CAST pipeline to the certificate theorems. Each major step in the CAST and CAST-conv+perm pipelines has a corresponding certificate-side motivation in the deterministic mask-certificate framework of Section **??**. Table 18 pairs each implementation component with the theorem or lemma that motivates it, and identifies the certificate quantity that the component is designed to improve.

Cross-architecture pre-FT behaviour: ViT vs. ResNet vs. ConvNeXt. The post-projection (pre-FT) top-1 of the second row of Table 17 shows architecture-dependent sensitivity to the 2:4 hardware constraint. For ViT-B with CAST, post-projection top-1 is 66.24% (recovering to 83.41% after 3 epochs of distillation FT, a +17.2 pp recovery). For DeiT-B (smaller than ViT-B but same Linear-only graph) the corresponding numbers are 70.83→80.48% (+9.7 pp recovery). In contrast, for ResNet50d the post-projection (pre-FT) top-1 collapses to 5.92% before recovering to 78.08% under the same 3-epoch distillation FT (a +72.2 pp recovery from a near-random starting point). This pattern is consistent across the ResNet family: the 2:4 constraint applied to the 1×1 + 3×3 conv weights causes a near-total functional collapse that is only undone by the FT phase, whereas ViT/DeiT Linear layers tolerate the same constraint with a much milder pre-FT drop and a smaller proportional FT-recovery contribution. Two complementary mechanisms are likely at play: (i) the bottleneck convs in a ResNet have heavily skewed weight distributions in which the 2-of-4 keep set is very different from the dense top-2 magnitude pattern, so the projection step replaces effective filters by a misaligned structurally legal 2:4 subset; (ii) ResNets lack the multi-head redundancy of attention, so a sparsely sampled feature map cannot be compensated for by neighbouring heads at inference time and must be recovered by re-training under the constraint. For the ResNet rows, the post-FT accuracy is therefore driven mainly by the FT phase under a 2:4-legal starting point, and CAST's role is to choose that starting point by per-row certificate cost rather than by magnitude alone. The CAST-vs-magnitude gap on ResNets is evaluated in the FT-finishing rows of Table 17.

17 Comparison with prior pruning methods on ViT-B/16

For ViT-B/16, CP-ViT [29] reports results on a checkpoint with baseline top-1 77.91%, whereas our checkpoint `vit_base_patch16_224.augreg2_in21k_ft_in1k` has baseline 85.11%. CP-ViT scans

FLOP reductions in the 15%–32.5% range and reports accuracy reductions in the 0.5–2.5 percentage-point range. Because the checkpoints differ, the comparison is indicative rather than apples-to-apples; Table 19 records it as a literature reference alongside our Hybrid Magnitude–SER numbers from Table **??** at the same compression range.

Table interpretation. At low-to-moderate compression ($\sim 30\%$), Hybrid Magnitude–SER on the AugReg-trained ViT-B/16 checkpoint achieves a 0.47 pp drop, comparable to or smaller than CP-ViT's reported ~ 0.9–1.5 pp range on its (worse) ViT-B/16 checkpoint. At higher compression where CP-ViT did not report numbers, Hybrid Magnitude–SER and SER preserve substantial accuracy at 50% parameter sparsity: 83.37% and 83.27%, respectively (Table **??**). At 70%, Hybrid remains at 78.01%, improving over classical magnitude by 10.57 pp and over classical RMT by 6.48 pp.

18 Comparison with prior pruning methods on DeiT

The DeiT family is the most-pruned ViT architecture in the literature, so it provides a natural head-to-head benchmark. Table 20 compares Hybrid Magnitude–SER against four representative DeiT pruning baselines reported by CP-ViT [29]: VTP [40], PoWER-BERT [14], HVT [25], and CP-ViT itself.

Three protocol caveats apply. First, VTP, PoWER-BERT, HVT, and CP-ViT use *structured* pruning (token reorganization, attention-head sparsity, hierarchical pooling), so their reported compression in the "Compression" column is FLOPs savings; Hybrid Magnitude–SER uses *unstructured* parameter pruning, so its compression is parameter sparsity. These coincide as a wall-clock speedup only when sparse kernels are used. Second, CP-ViT and the reproduced VTP/PoWER/HVT numbers fine-tune for 30 ImageNet epochs (CP-ViT §4 Implementation Details); Hybrid Magnitude–SER fine-tunes for one epoch per sparsity step, totaling 4 epochs at $s = 0.20$ and 7–8 epochs at $s = 0.40$. Third, Hybrid Magnitude–SER values come from the multi-architecture sweep of Table 16, which uses a single cross-architecture set of hyperparameters; the prior methods were tuned per checkpoint.

Reading the table. At low compression ($\sim 20\%$), Hybrid Magnitude–SER has a smaller reported top-1 drop than the listed baselines on each DeiT model: -0.35 versus -0.96 for CP-ViT on DeiT-Tiny, -0.48 versus -0.72 on DeiT-Small, -0.34 versus -0.69 on DeiT-Base. At higher compression ($\sim 40\%$), the comparison is more nuanced: on DeiT-Small, Hybrid Magnitude–SER (-1.24 at 40% parameter sparsity) is between PoWER (-1.50) and CP-ViT (-0.72), and on DeiT-Base the closest available cell ($s = 0.35$, -0.63) is comparable to CP-ViT's -0.69 at slightly higher compression. Hybrid Magnitude–SER reports *parameter sparsity* from unstructured masks, whereas CP-ViT reports *FLOPs savings* from structured pruning; a direct numerical match at the same percentage therefore masks a protocol difference. The low-compression observation is that a single cross-architecture hybrid recipe has a smaller reported top-1 drop than the listed DeiT-tuned structured baselines in the 20%-parameter-sparsity rows, subject to the protocol caveats above.

This section presents the abstract additive perturbation extension from Main-§4 ("Abstract Additive Perturbation Extensions"). It generalizes the Gaussian sufficient-condition framework of Main-Appendix A to non-Gaussian random components by replacing iid Gaussian concentration bounds with abstract conditions on the propagated logit perturbation, persistent first-order cross terms in the regularizer, and one-sided stationarity in the denoising direction. The deterministic mask certificate of Main-§5 is independent of this material.

19 Abstract Additive Perturbation Extensions

This section presents the abstract additive perturbation counterpart to the deterministic mask analysis. It interprets random-like components in stylized additive models, while the unstructured pruning operation is covered by the mask certificates in Section **??**. The statements below complement the mask certificates by showing how the same propagated-logit mechanism behaves for additive decompositions $W = S + R$. For a concrete Gaussian specialization and the associated spectral consequences, see Appendix **??**. Each theorem or corollary points to its proof in Appendices **??**–**??**.

The correspondence with the Gaussian/RMT appendix is as follows. Lemma 19.9 generalizes the Gaussian logit perturbation bound of Lemma **??** and the confidence version in Theorem **??**. Theorem 19.10 generalizes the Gaussian loss-reduction theorem, Theorem **??**. Corollary 19.13 generalizes the Gaussian pathwise denoising bound, Corollary **??**. Theorem 19.16 abstracts the Gaussian stationarity-collapse theorem, Theorem **??**, whose MP-scale consequences are Corollaries **??**–**??**. Finally, Corollary 19.18 is the general inverse-D analogue of the square Gaussian BBP statement in Corollary **??**.

19.1 Assumptions for the generalized perturbation framework

Throughout the asymptotic statements in this section, the training set T is finite and fixed independently of N, unless a result explicitly states summability conditions for a growing family.

Assumption 19.1 (General architecture)**.** Assume the DNN can be written as

$$\phi = \rho \circ \psi_2 \circ (R + S) \circ \psi_1,$$

where ψ_1, ψ_2 may depend on the remaining network parameters and ρ is softmax. For each $s \in T$, let $\mathcal{U}_s$ be any set containing the admissible interpolation segment $\{(S + \vartheta R)\psi_1(s) : \vartheta \in [0, 1]\}$. We assume that ψ_2 has a finite local Lipschitz constant with respect to ℓ_∞ on $\mathcal{U}_s$:

$$L_{\psi_2}(s) := \sup_{v \neq w} \frac{\|\psi_2(v) - \psi_2(w)\|_\infty}{\|v - w\|_\infty} < \infty.$$

Here the supremum is taken only over $\mathcal{U}_s$, so only a local-on-the-path Lipschitz bound is required.

Remark 19.2 (Role of Assumption 19.1). This assumption primarily fixes notation. After freezing all other weights, the network is factored around the layer being studied: $\psi_1(s)$ is the input activation to that layer, $R + S$ is the layer matrix, and ψ_2 maps the layer output to logits. The Lipschitz constant is required only on the finite path traced by the interpolation $S + \vartheta R$, not globally on the whole ambient space. For ReLU or piecewise-linear networks this is automatic on bounded paths; for architectures with normalization layers it is an explicit local regularity condition, excluding only pathological paths where the post-layer map becomes singular. In the Gaussian model of Appendix **??**, Assumption **??** is the concrete three-layer instance of this factorization with the target layer $W_2 = S_2 + R_2$.

Assumption 19.3 (Perturbative random component)**.** Assume that for every deterministic vector v, and more generally for every v measurable with respect to the conditioning variables $\sigma(S, \psi_1, \psi_2)$,

$$\mathbb{P}\Big(\|Rv\|_\infty > d_1(N)J(v) \,\Big|\, S, \psi_1, \psi_2\Big) \leq d_2(N), \tag{17}$$

where $d_1(N), d_2(N) \to 0$ and $J(v)$ depends only on v. When a spectral interpretation is desired, we additionally assume that the singular-value ESD of R converges to a deterministic law with finite right edge $b_+ < \infty$.

Remark 19.4 (Role of Assumption 19.3). The vector v is fixed before the randomness in R is revealed. In the network application $v = \psi_1(s)$, so the pre-layer activation is part of the conditioning information. The assumption does not require Gaussian entries; it requires only that every data activation is sent by R to a small vector in ℓ_∞ norm, with high probability. The optional empirical-spectral-distribution condition is used only for RMT interpretation and spectral corollaries. The loss and certificate bounds use the concentration estimate (17), not the ESD convergence.

The Gaussian connection is explicit: Assumption **??** in Appendix **??** implies Assumption 19.3 with $J(v) = \|v\|_2$ and $d_1(N)$ of order $\sqrt{\log N/N}$. The confidence-parameter version is Theorem **??**; the MP-edge interpretation of the same variance scale is Corollary **??**.

Example 19.5. If R has i.i.d. Gaussian entries with variance $1/N$, then

$$\mathbb{P}\left(\|Rv\|_\infty > 2\sqrt{2}\,\|v\|_2 \sqrt{\frac{\log(2N)}{N}} \right) \le \frac{1}{N},$$

so Assumption 19.3 holds with $d_1(N) = 2\sqrt{2\log(2N)/N}$, $d_2(N) = 1/N$, and $J(v) = \|v\|_2$. The concrete three-layer Gaussian specialization is Lemma **??** and Theorem **??** in Appendix **??**; the proof of the abstract concentration step is in Appendix **??**. The same conditional formulation also accommodates sub-Gaussian rows, orthogonally invariant perturbations, or moderate training-induced dependence, provided the displayed concentration estimate survives.

Example 19.6 (Sub-Gaussian rows)**.** Assume, conditionally on S, ψ_1, ψ_2, that the rows $r_1, \dots, r_N$ of R are independent, centered, and satisfy

$$\|\langle r_j, v\rangle\|_{\psi_2} \le \frac{\kappa}{\sqrt{N}}\|v\|_2 \qquad \text{for all } v \text{ and all } j.$$

Then there is a universal constant C such that, for every $\delta \in (0,1)$,

$$\mathbb{P}\left(\|Rv\|_\infty > C\kappa\|v\|_2 \sqrt{\frac{\log(2N/\delta)}{N}} \,\middle|\, S, \psi_1, \psi_2 \right) \le \delta.$$

Thus the abstract framework is not intrinsically Gaussian; it requires only row-wise concentration at the $N^{-1/2}\sqrt{\log N}$ scale.

Assumption 19.7 (Structured component for optional spectral conclusions)**.** Whenever a spectral conclusion is invoked, assume that there exists a sequence $k_N = o(\min\{N, M\})$ such that

$$\sum_{i > k_N} s_i(S)^2 \to 0.$$

The convergence is in probability when S is random.

Remark 19.8 (Role of Assumption 19.7). This assumption is used only when the paper draws spectral conclusions about the structured part S. It is not needed for the loss-reduction theorems, the data-path certificate, or the stationarity-collapse inequality. The condition means that all but $o(\min\{N, M\})$ singular directions of S carry vanishing squared singular-value mass.

Exact finite rank is the simplest case. If $\operatorname{rank}(S) \le r_0$ for a constant r_0, take $k_N = r_0$. Then $k_N = o(\min\{N, M\})$ and $\sum_{i>k_N} s_i(S)^2 = 0$, so Assumption 19.7 holds automatically. More generally, if $\operatorname{rank}(S) = r_N = o(\min\{N, M\})$, take $k_N = r_N$. Approximate low rank allows a small tail: the rank may not be exactly finite, but the singular-value energy beyond k_N must vanish. The Gaussian analogue is Assumption **??** in Appendix **??**.

For a DNN satisfying Assumption 19.1, define

$$h_\phi(s) := J(\psi_1(s))\, L_{\psi_2}(s). \tag{18}$$

The remaining hypotheses appearing in the theorems are quantitative smallness conditions rather than structural assumptions. The condition $a_\phi(N,s) = h_\phi(s)d_1(N) \to 0$ requires the random-like component to be weak after propagation through the data path. The condition $\langle S, R\rangle_F = o_{\mathbb{P}}(1)$ requires the regularizer to have no persistent first-order cross term when R is removed. In the Gaussian appendix, Assumption **??** supplies the corresponding data-path scaling and Assumption **??** supplies the cross-term control. Finally, the one-sided stationarity hypotheses in Theorems 19.16 and **??** are not random-matrix assumptions; they require the trained model to be nearly stationary in the particular denoising direction being tested.

19.2 Generalized perturbation and loss-reduction results

We consider the regularized loss

$$L(\alpha(t)) = -\frac{1}{|T|}\sum_{s\in T} \log\left(\phi_{C(s)}(s,\alpha(t))\right) + \lambda_{\text{reg}} \sum_{i=1}^{L} \|W_i(t)\|_F^2. \tag{19}$$

Lemma 19.9. *Let*

$$X = \psi_2 \circ (R+S) \circ \psi_1(s),$$

and let α_W, α_S denote the corresponding network parameters when the relevant weight matrix is $W = R+S$ and S, respectively. Under Assumptions 19.1–19.3,

$$\mathbb{P}\Big(\|X(s,\alpha_S) - X(s,\alpha_W)\|_\infty \le d_1(N)h_\phi(s)\Big) \ge 1 - d_2(N).$$

*Proof location: See Appendix **??**, Proof of Lemma 19.9.*

Theorem 19.10. *Assume Assumptions 19.1–19.3 and that*

$$\langle S, R\rangle_F \to 0 \qquad \text{in probability as } N\to\infty.$$

Assume further that for all $s \in T$,

$$a_\phi(N,s) := h_\phi(s)d_1(N) \to 0 \qquad \text{as } N\to\infty.$$

Then removing R yields

$$L(\alpha_S) = L(\alpha_W) - \lambda_{\text{reg}}\|R\|_F^2 + o_{\mathbb{P}}(1) \qquad (N\to\infty).$$

*Proof location: See Appendix **??**, Proof of Theorem 19.10.*

In the square three-layer Gaussian model, Theorem **??** in Appendix **??** gives the corresponding finite-N loss-reduction statement, and Theorem **??** gives the confidence-parameter version.

Corollary 19.11 (Growing-set abstract loss reduction)**.** *Let T_N be a finite labeled set that may depend on N. Let L_{reg,T_N} denote cross-entropy averaged over T_N, and let L_{T_N} denote the corresponding regularized objective with the same Frobenius penalty as in (19). Assume Assumptions 19.1–19.3,*

$$\langle S, R\rangle_F = o_{\mathbb{P}}(1), \qquad |T_N| d_2(N) \to 0,$$

and

$$\frac{1}{|T_N|}\sum_{s\in T_N} a_\phi(N,s) \to 0$$

in probability. Then

$$L_{T_N}(\alpha_S) = L_{T_N}(\alpha_W) - \lambda_{\text{reg}}\|R\|_F^2 + o_{\mathbb{P}}(1).$$

*Proof location: See Appendix **??**, Proof of Corollary 19.11.*

Corollary 19.12. *Under the hypotheses of Theorem 19.10, for any fixed $\vartheta \in [0,1]$, define*

$$W(\vartheta) = S + \vartheta R.$$

Then

$$L(\alpha_{W(\vartheta)}) = L(\alpha_W) - \lambda_{\mathrm{reg}}(1-\vartheta^2)\|R\|_F^2 + o_{\mathbb{P}}(1) \qquad (N \to \infty).$$

The same asymptotic statement holds along any deterministic sequence $\vartheta_N \in [0,1]$.

*Proof location: See Appendix **??**, Proof of Corollary 19.12.*

Corollary 19.13 (Finite-N scaled loss bound along the denoising path)**.** *Assume the hypotheses of Theorem 19.10, and define $W(\vartheta) = S + \vartheta R$ for $\vartheta \in [0,1]$. Then for every $\eta > 0$ there exists an event $E_{N,\eta}$ with*

$$\mathbb{P}(E_{N,\eta}) \geq 1 - |T| d_2(N) - \mathbb{P}\big(|\langle S, R\rangle_F| > \eta\big)$$

such that on $E_{N,\eta}$,

$$L(\alpha_{W(\vartheta)}) \leq L(\alpha_W) - \lambda_{\mathrm{reg}}(1-\vartheta^2)\|R\|_F^2 + (1-\vartheta)\frac{2}{|T|}\sum_{s\in T} a_\phi(N,s) + 2\lambda_{\mathrm{reg}}(1-\vartheta)\eta$$

for every $\vartheta \in [0,1]$.

*Proof location: See Appendix **??**, Proof of Corollary 19.13.*

The Gaussian pathwise analogue is Corollary **??** in Appendix **??**; it is the specialization used in the Gaussian stationarity-collapse theorem.

Theorem 19.14 (Multi-layer telescoping loss reduction)**.** *Let $\mathcal{P}$ be a finite set of layers selected for pruning, with $|\mathcal{P}|$ fixed independently of N. For each $\ell \in \mathcal{P}$, write*

$$W_\ell = S_\ell + R_\ell,$$

and assume that, after the previous layers in $\mathcal{P}$ have already been replaced by their structured parts, the single-layer hypotheses of Theorem 19.10 continue to hold for layer ℓ with perturbation scale $a_{\phi,\ell}(N,s)$. If $\alpha^{(0)}$ denotes the original network parameters and $\alpha^{(|\mathcal{P}|)}$ the parameters after replacing every W_ℓ by S_ℓ, then

$$L(\alpha^{(|\mathcal{P}|)}) = L(\alpha^{(0)}) - \lambda_{\mathrm{reg}} \sum_{\ell\in\mathcal{P}} \|R_\ell\|_F^2 + o_{\mathbb{P}}(1).$$

Thus the one-layer denoising theorem extends to the stylized multi-layer pruning pipeline by telescoping the loss differences over the pruned layers.

More generally, the set of pruned layers may depend on N. Let $\mathcal{P}_N$ be a sequence of layer sets, ordered in the pruning order. Assume that the corresponding one-layer logit events $E_{\ell,N}$ satisfy

$$\sum_{\ell\in\mathcal{P}_N} \mathbb{P}(E_{\ell,N}^c) \to 0,$$

that their perturbation scales are summable,

$$\sum_{\ell\in\mathcal{P}_N} \frac{1}{|T|}\sum_{s\in T} a_{\phi,\ell}(N,s) \to 0,$$

and that the accumulated cross term is negligible,

$$\sum_{\ell\in\mathcal{P}_N} \langle S_\ell, R_\ell\rangle_F = o_{\mathbb{P}}(1).$$

Then the same telescoping conclusion holds with $\mathcal{P}$ replaced by $\mathcal{P}_N$.

Proof location: See Appendix **??**, *Proof of Theorem 19.14.*

Remark 19.15 (Beyond pure Frobenius regularization). The quadratic penalty in (19) is used only to obtain a coercive lower bound of order $\|R\|_F^2$. For a general differentiable μ-strongly convex regularizer Ω, one has the exact decomposition

$$\Omega(S+R) - \Omega(S) = \langle \nabla\Omega(S), R\rangle + D_\Omega(S+R, S),$$

where

$$D_\Omega(S+R, S) \geq \frac{\mu}{2}\|R\|_F^2.$$

Thus the same extension is valid provided the first-order term $\langle \nabla\Omega(S), R\rangle$ is controlled or negligible along the admissible sequence. In the Frobenius case $\Omega(W) = \|W\|_F^2$, this term is exactly $2\langle S, R\rangle_F$, which is already part of the hypotheses. For the elastic-net penalty

$$\Omega(W) = \mu_1\|W\|_1 + \mu_2\|W\|_F^2, \qquad \mu_2 > 0,$$

the L_1 term is nonsmooth and can increase when R is removed if R cancels entries of S. The correct extension is subgradient-based: choose $\xi \in \partial\|S\|_1$ and assume

$$\langle \mu_1\xi + 2\mu_2 S, R\rangle = o_{\mathbb{P}}(1).$$

The quadratic part then supplies the strong convexity needed for the same type of loss-reduction bound.

Theorem 19.16. *Assume Assumptions 19.1–19.3. Assume further that for all $s \in T$,*

$$a_\phi(N, s) := h_\phi(s) d_1(N) \to 0 \qquad \text{as } N \to \infty.$$

Suppose:

1. *for each N, $\alpha(t, N) \to \alpha^*(N)$ in probability as $t \to \infty$;*
2. *the relevant limit weight matrix admits an admissible decomposition $W^*(N) = S^*(N) + R^*(N)$ satisfying*
$$\langle S^*(N), R^*(N)\rangle_F \to 0 \qquad \text{in probability as } N \to \infty.$$

For $\vartheta \in [0,1]$, let $\alpha^*_\vartheta(N)$ denote the parameter vector obtained by replacing $W^*(N)$ with $S^*(N)+\vartheta R^*(N)$. Assume further that there exists a sequence of nonnegative random variables $\epsilon_N = o_{\mathbb{P}}(1)$ such that the one-sided directional stationarity event

$$\liminf_{h\downarrow 0} \frac{L(\alpha^*_{1-h}(N)) - L(\alpha^*(N))}{h} \geq -\epsilon_N$$

has probability tending to one. Define

$$\bar{a}_{\phi,N} := \frac{2}{|T|}\sum_{s\in T} a_\phi(N, s).$$

Then

$$\|R^*(N)\|_F^2 \leq \frac{\bar{a}_{\phi,N} + \epsilon_N}{2\lambda_{\text{reg}}} + o_{\mathbb{P}}(1).$$

In particular, if $\bar{a}_{\phi,N} + \epsilon_N \to 0$ in probability, then for every $\epsilon > 0$,

$$\lim_{N\to\infty} \mathbb{P}\big(\|R^*(N)\|_F \leq \epsilon\big) = 1.$$

If, in addition, there exists an admissible path decomposition

$$W(t,N) = S(t,N) + R(t,N)$$

such that for each fixed N,

$$\|S(t,N) - S^*(N)\|_F + \|R(t,N) - R^*(N)\|_F \to 0 \qquad \text{in probability as } t \to \infty,$$

and if for every fixed N and every $\epsilon > 0$,

$$\mathbb{P}\big(\|R^*(N)\|_F = \epsilon\big) = 0,$$

then

$$\lim_{N\to\infty} \lim_{t\to\infty} \mathbb{P}\big(\|R(t,N)\|_F \le \epsilon\big) = 1.$$

Proof location: See Appendix **??**, *Proof of Theorem 19.16.*

Appendix **??** gives the concrete Gaussian/RMT version of this conclusion: Theorem **??** proves collapse of the admissible Gaussian component, Corollary **??** translates this into collapse of the MP variance scale, and Corollary **??** records the resulting spectral stabilization.

Remark 19.17. The theorem assumes only one-sided directional approximate stationarity at the unpruned point $\vartheta = 1$. Exact local minimality is a sufficient special case with $\epsilon_N \equiv 0$, but the hypothesis is weaker than a global lower bound along the whole denoising path. Without the continuity assumption $\mathbb{P}(\|R^*(N)\|_F = \epsilon) = 0$, the same proof still gives the weaker limsup/liminf bounds obtained from Portmanteau-type arguments, but not necessarily exact convergence of the probabilities at the threshold ϵ.

Corollary 19.18 (Eventual supercriticality of persistent spike clusters in a BGN-type deformation regime)**.** *Under the hypotheses of Theorem 19.16, assume additionally that for each N the pair $W^*(N) = S^*(N)+R^*(N)$ lies in a finite-rank deformation regime satisfying all hypotheses of Theorem* **??** *for the spikes considered below. In particular, assume the regime has a deterministic bulk edge $b_+(N)$, a deterministic critical threshold $\theta_c(N)$, a strictly decreasing outlier transform*

$$D_N : (b_+(N), \infty) \to (0, D_N(b_+(N)^+))$$

in the sense of Appendix **??**, *local Lipschitz/invertibility intervals around the outliers, and concentration of the relevant sample singular values in those intervals. Assume that*

$$\theta_c(N) \to 0, \qquad b_+(N) \to 0.$$

Assume further that for some fixed $k \ge 1$ there exist constants

$$\bar\sigma_1 > \cdots > \bar\sigma_k > 0$$

such that

$$\sigma_i^*(N) \to \bar\sigma_i \qquad \text{in probability as } N \to \infty$$

for each $1 \le i \le k$. Then:

1. $$\mathbb{P}\big(\sigma_i^*(N) > \theta_c(N)\big) \to 1 \qquad \text{for each } 1 \le i \le k;$$

2. *these k spikes are eventually supercritical, so the corresponding singular values of $W^*(N)$ form outlier clusters above the edge $b_+(N)$. Moreover, if the i-th spike is separated from the other nonzero singular values of $S^*(N)$ by a gap $\Delta_i(N) > 0$ with*

$$\frac{\|R^*(N)\|_{\mathrm{op}}}{\Delta_i(N)} \to 0 \qquad \textit{in probability,}$$

then the associated rank-one left and right spectral projectors converge in operator norm to the corresponding projectors of $S^*(N)$;

3. *defining the inverse-D estimator on the event $s_i(W^*(N)) > b_+(N)$, whose probability tends to one, by*

$$\widehat{\sigma}_i^{(D)}(N) := D_N(s_i(W^*(N)))^{-1/2},$$

and defining it arbitrarily off that event, one has

$$\widehat{\sigma}_i^{(D)}(N) - \sigma_i^*(N) \to 0 \qquad \textit{in probability}$$

for each $1 \le i \le k$.

*Proof location: See Appendix **??**, Proof of Corollary 19.18.*

For the square Gaussian finite-rank case, the corresponding RMT statement is Corollary **??** in Appendix **??**; the external BGN input and the inverse-D abstraction are stated in Appendix **??**.

Remark 19.19. Corollary 19.18 is the abstract analogue of the Gaussian BBP statement. It says that once a finite-rank deformation model has a well-defined critical threshold $\theta_c(N)$, vanishing admissible randomness forces every structurally persistent spike cluster to cross that threshold eventually. Thus the limit theory does not merely predict collapse of the random bulk; it predicts a detectability transition for persistent informative directions in any deformation regime governed by an inverse-D outlier map.

Remark 19.20 (Assumption map for the abstract and deterministic theory). The following list summarizes which assumptions are used by the abstract additive results in this section and by the deterministic mask results in Section **??**:

1. Lemma **??**, Corollaries **??**, **??**, **??**, **??**, and **??**, and Theorems **??**–**??** are deterministic statements. They do not assume an additive Gaussian decomposition; the mask-specific results use only disjoint support of the kept, restored, and removed entries.

2. Corollaries **??**–**??** give random sufficient conditions for the deterministic data-path budgets.

3. Lemma 19.9 uses only Assumptions 19.1–19.3.

4. Theorem 19.10, Corollary 19.12, and the finite-N pathwise Corollary 19.13 use Assumptions 19.1–19.3 together with the additional hypotheses $\langle S, R\rangle_F = o_{\mathbb{P}}(1)$ and $a_\phi(N, s) \to 0$ for each $s \in T$. Assumption 19.7 is not needed for these loss bounds. Their concrete Gaussian counterparts are Theorem **??** and Corollary **??** in Appendix **??**.

5. Theorem 19.14 is obtained by applying Theorem 19.10 layer by layer; the second part allows a growing layer set under summable perturbation and failure-probability conditions.

6. Theorem 19.16 uses Assumptions 19.1–19.3, the same scaling condition $a_\phi(N, s) \to 0$, the cross-term condition at the limit point, and the one-sided directional stationarity hypothesis stated in the theorem. Its Gaussian counterpart is Theorem **??**, with MP variance and spectral consequences in Corollaries **??**–**??**.

7. Corollary 19.18 further assumes that the pair $W^*(N) = S^*(N) + R^*(N)$ satisfies the finite-rank deformation hypotheses of Theorem **??**, a threshold $\theta_c(N) \to 0$, an edge $b_+(N) \to 0$, and a persistent-spike hypothesis $\sigma_i^*(N) \to \bar{\sigma}_i > 0$. The square Gaussian BBP specialization is Corollary **??**.

8. The pathwise conclusion in Theorem 19.16 additionally uses the path-decomposition convergence assumption, while the no-atom condition $\mathbb{P}(\|R^*(N)\|_F = \epsilon) = 0$ is only needed for exact convergence of the probabilities at the threshold ϵ.

Acknowledgments

LB, HO and YS acknowledge support from NASA via the AIST program (Kernel Flows: Emulating Complex Models for Massive Data Sets). This work started when LB was on a sabbatical stay at Caltech hosted by H. Owhadi. Both LB and YS are grateful for the hospitality during their visit, supported by NASA.

The work of LB was partially supported by NSF grant DMS-2005262 and NSF grant IMPRESS-U 2401227. TB and HO acknowledge support from the Air Force Office of Scientific Research under MURI award number FA9550-20-1-0358 (Machine Learning and Physics-Based Modeling and Simulation), FOA-AFRL-AFOSR-2023-0004 (Mathematics of Digital Twins), and by the Department of Energy under award number DE-SC0023163 (SEA-CROGS: Scalable, Efficient, and Accelerated Causal Reasoning Operators, Graphs and Spikes for Earth and Embedded Systems). Additionally, HO and TB acknowledge support from the DoD Vannevar Bush Faculty Fellowship Program under award number ONR-N000142512035.

We additionally thank the maintainers of timm [36], PyTorch [26], and the HuggingFace Hub for the open-source infrastructure on which the experiments in this paper depend. The 2:4 native sparse Tensor Core kernel is provided by NVIDIA [24]; the ToMe token merging method is by Bolya et al. [7].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used generative AI tools to accelerate drafting and revision. All mathematical claims, proofs, numerical interpretations, and bibliographic information were subsequently reviewed and edited by the author(s), who take full responsibility for the content of the manuscript.

References

[1] et al. An. SERo: Sparse and efficient reordering for vision transformers. In *UAI / PMLR*, 2025.

[2] (arXiv 2407.02068). LPViT: Low-power layer-pruned vision transformer. *arXiv preprint arXiv:2407.02068*, 2024.

[3] (arXiv 2409.09708). ELSA: Efficient layer-wise sparse adapters for vision transformer. *arXiv preprint arXiv:2409.09708*, 2024.

[4] (arXiv 2410.16135). Beyond 2:4: Exploring V:N:M sparsity for hardware-accelerated inference. *arXiv preprint arXiv:2410.16135*, 2024.

[5] Hangbo Bao, Li Dong, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations*, 2022.

[6] Leonid Berlyand, Etienne Sandier, Yitzchak Shmalo, and Lei Zhang. Enhancing accuracy in deep learning using random matrix theory. *Journal of Machine Learning*, 3(4):347–412, 2024. doi: 10.4208/jml.231220.

[7] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=JroZRaRw7Eu.

[8] Chun-Fu Richard Chen, Quanfu Fan, and Rameswar Panda. CrossViT: Cross-attention multi-scale vision transformer for image classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 357–366, 2021.

[9] Zihang Dai, Hanxiao Liu, Quoc V. Le, and Mingxing Tan. CoAtNet: Marrying convolution and attention for all data sizes. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[10] Mingyu Ding, Bin Xiao, Noel Codella, Ping Luo, Jingdong Wang, and Lu Yuan. DaViT: Dual attention vision transformers. In *European Conference on Computer Vision*, pages 74–92. Springer, 2022.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[12] Alaaeldin El-Nouby, Hugo Touvron, Mathilde Caron, Piotr Bojanowski, Matthijs Douze, Armand Joulin, Ivan Laptev, Natalia Neverova, Gabriel Synnaeve, Jakob Verbeek, and Herve Jegou. XCiT: Cross-covariance image transformers. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[13] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. EVA-02: A visual representation for neon genesis. *arXiv preprint arXiv:2303.11331*, 2023.

[14] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. Power-bert: Accelerating BERT inference via progressive word-vector elimination. In *International Conference on Machine Learning*, pages 3690–3699, 2020.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[16] Zheng Tracy Ke, Yucong Ma, and Xihong Lin. Estimation of the number of spiked eigenvalues in a covariance matrix by bulk eigenvalue matching analysis. *Journal of the American Statistical Association*, 117(538):962–974, 2022.

[17] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. MViTv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022.

[18] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021.

[19] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12009–12019, 2022.

[20] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022.

[21] Haiquan Lu, Yefan Zhou, Shiwei Liu, Zhangyang Wang, Michael W. Mahoney, and Yaoqing Yang. Alphapruning: Using heavy-tailed self regularization theory for improved layer-wise pruning of large language models. *Advances in Neural Information Processing Systems*, 2024.

[22] Charles H. Martin and Michael W. Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In *Proceedings of the 2020 SIAM International Conference on Data Mining*, pages 505–513, 2020.

[23] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.

[24] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021. doi: 10.48550/arXiv.2104.08378.

[25] Zizheng Pan, Bohan Zhuang, Jing Liu, Haoyu He, and Jianfei Cai. Scalable vision transformers with hierarchical pooling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 377–386, 2021.

[26] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[27] Chaitanya Ryali, Yuan-Ting Hu, Daniel Bolya, Chen Wei, Haoqi Fan, Po-Yao Huang, Vaibhav Aggarwal, Arkabandhu Chowdhury, Omid Poursaeed, Judy Hoffman, Jitendra Malik, Yanghao Li, and Christoph Feichtenhofer. Hiera: A hierarchical vision transformer without the bells-and-whistles. In *Proceedings of the 40th International Conference on Machine Learning*, pages 29441–29454, 2023.

[28] Yitzchak Shmalo, Jonathan Jenkins, and Oleksii Krupchytskyi. Deep learning weight pruning with rmt-svd: Increasing accuracy and reducing overfitting. *arXiv preprint arXiv:2303.08986*, 2023.

[29] Zhuoran Song, Yihong Xu, Zhezhi He, Li Jiang, Naifeng Jing, and Xiaoyao Liang. Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction. *arXiv preprint arXiv:2203.04570*, 2022.

[30] Max Staats, Matthias Thamm, and Bernd Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. *Physical Review E*, 108(2):L022302, 2023. doi: 10.1103/PhysRevE.108.L022302.

[31] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research*, 2022. URL https://openreview.net/forum?id=4nPswr1KcP.

[32] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021.

[33] Hugo Touvron, Matthieu Cord, Alaaeldin El-Nouby, Jakob Verbeek, and Herve Jegou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 32–42, 2021.

[34] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan C. Bovik, and Yinxiao Li. MaxViT: Multi-axis vision transformer. In *European Conference on Computer Vision*, pages 459–479. Springer, 2022.

[35] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. PVT v2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8:415–424, 2022.

[36] Ross Wightman. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.

[37] Jingjing Xie et al. UniPTS: A unified framework for proficient post-training sparsity. In *CVPR*, 2024.

[38] Chong Yu, Tao Chen, Zhongxue Gao, and Yongming Yan. Boost vision transformer with gpu-friendly sparsity and quantization. In *CVPR*, 2023.

[39] Li Yuan, Qibin Hou, Zihang Jiang, Jiashi Feng, and Shuicheng Yan. VOLO: Vision outlooker for visual recognition. *arXiv preprint arXiv:2106.13112*, 2021.

[40] Mingjian Zhu, Kai Han, Yehui Tang, and Yunhe Wang. Visual transformer pruning. *arXiv preprint arXiv:2104.08500*, 2021.

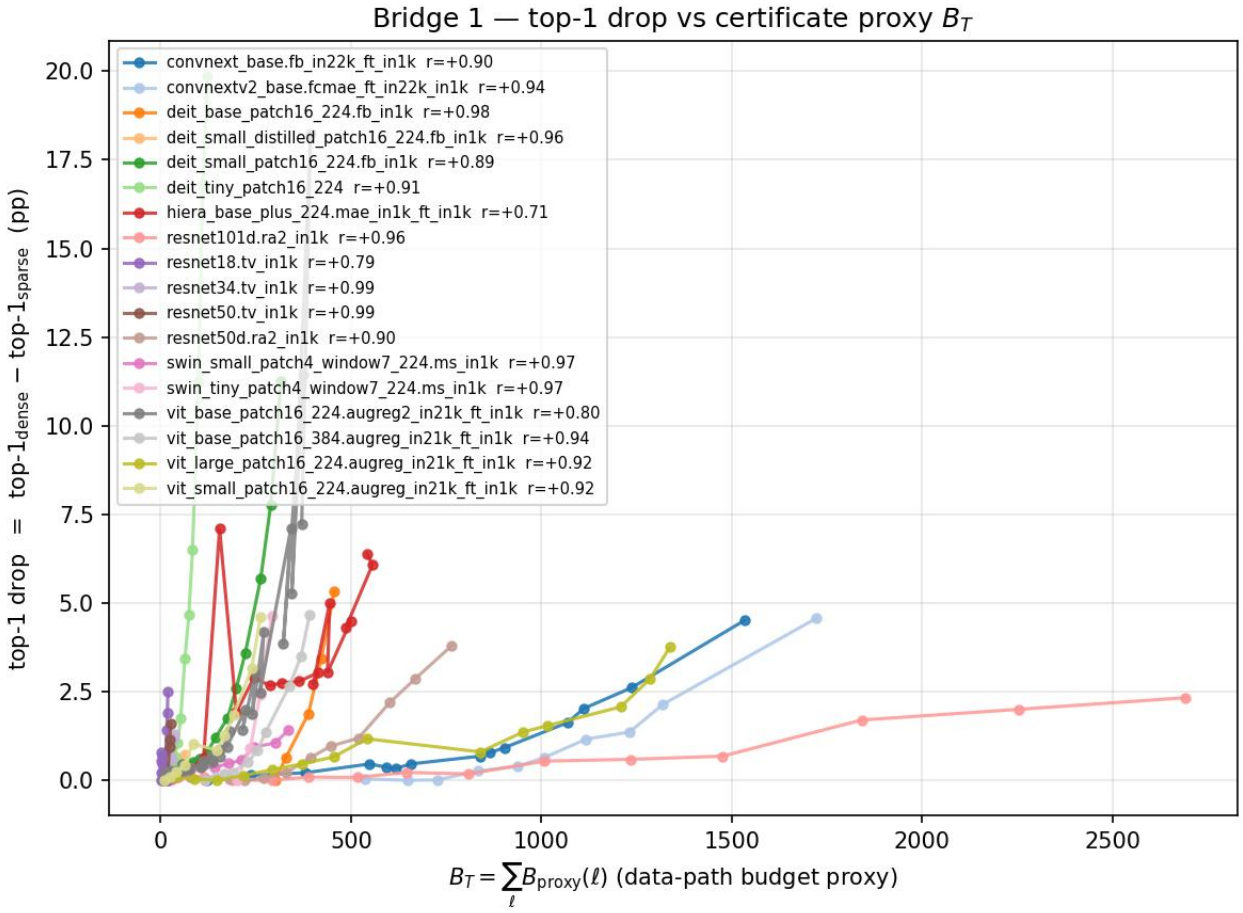


Figure 7: Top-1 drop and the operator-norm proxy $\hat{B}_T$ across all 18 paper architectures. Each curve is one architecture; each marker is one Hybrid Magnitude–SER cycle. Within-architecture Pearson correlation between the operator-norm proxy $\hat{B}_T = \sum_\ell \|R_\ell\|_{\text{op}} \overline{\|\psi_{1,\ell}\|_\infty}$ and the post-fine-tuning top-1 drop is positive for every architecture: r ranges from +0.71 (Hiera-Base+) to +0.99 (ResNet50/34 `tv_in1k`), with mean +0.91 over the 18 models. We report this as an empirical correlation: the proxy $\hat{B}_T$ is not the deterministic budget $B_T(R)$ of Eq. (see **??**) ($\|R\|_{\text{op}} \|\psi\|_\infty$ is not in general an upper bound for $\|R\psi\|_\infty$), so this correlation is suggestive of the certificate quantity's relevance but does not by itself verify Lemma **??**.

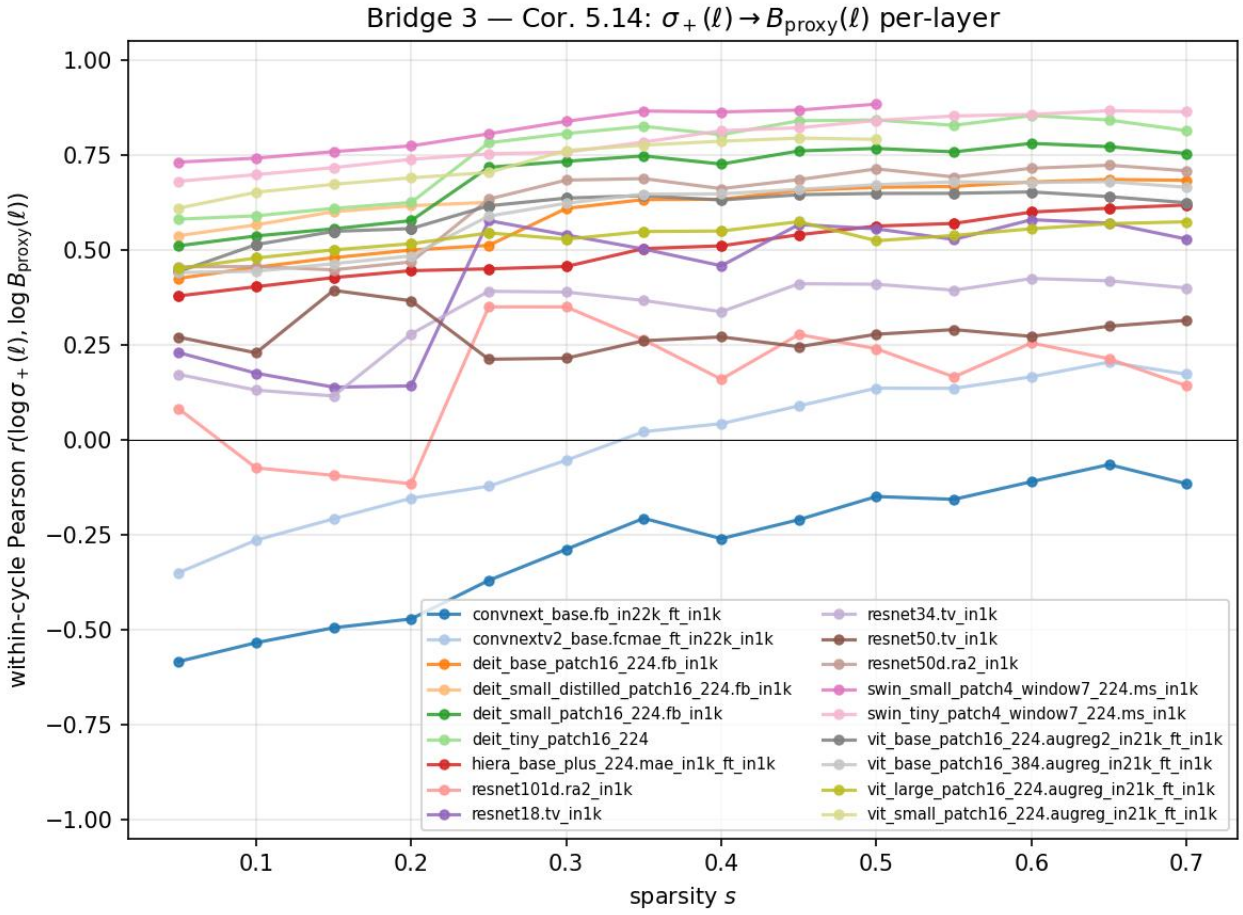


Figure 8: Within-cycle log-log Pearson correlation $r(\log \sigma_+(\ell), \log \hat{B}_{T,\ell})$ as a function of sparsity s, per architecture. Transformer architectures (Swin/DeiT/ViT) cluster in the +0.5 to +0.81 band, consistent with the iid-Gaussian heuristic of Corollary **??** (which is a sufficient condition under iid weights, not a certificate for the realized magnitude-selected mask). ResNet/Hiera families are positive but weaker (+0.16 to +0.62). The two ConvNeXt rows are the only architectural exception: ConvNeXtV2-Base averages ≈ 0 and ConvNeXt-Base averages -0.29. Per-layer-type stratification shows the ConvNeXt anomaly is concentrated in depthwise-convolution layers; ConvNeXt MLP layers behave like ViT MLP layers ($r = +0.51$ and $+0.58$ respectively).

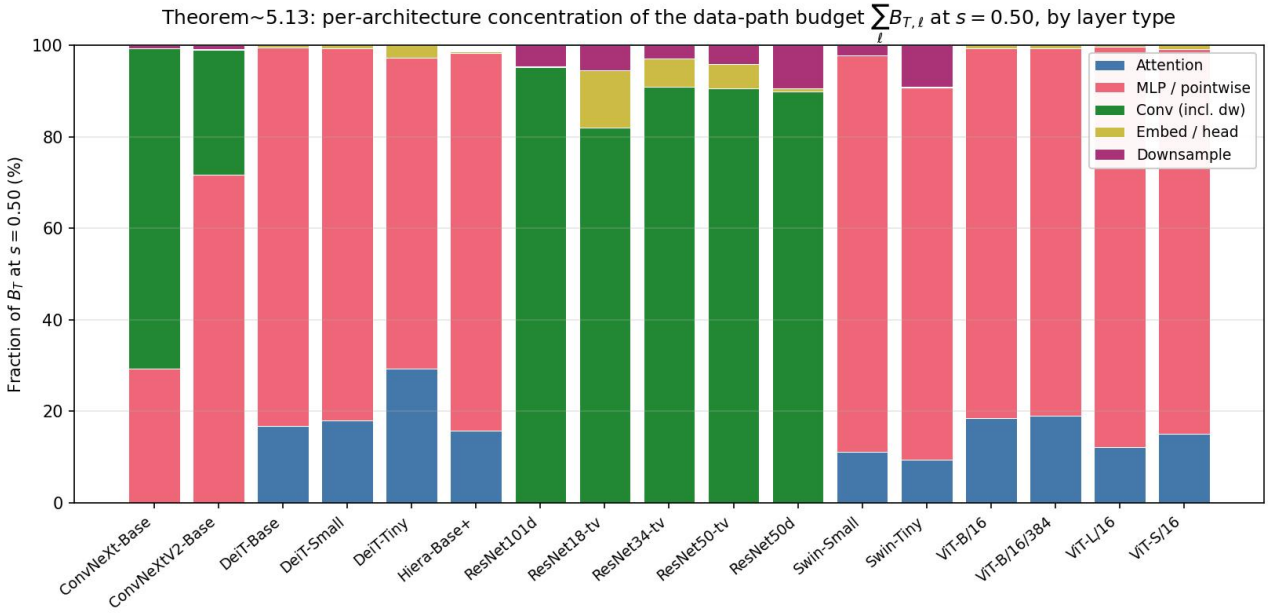


Figure 9: Per-architecture decomposition of $\sum_\ell \hat{B}_{T,\ell}$ at $s = 0.50$ by layer type. The bar height of each segment is its fraction of the total B_{total} for that architecture. Transformers (DeiT/Swin/ViT) are dominated by MLP/pointwise; ConvNeXt-Base is dominated by depthwise convolutions, ConvNeXtV2-Base by MLP/pointwise; ResNets by their convolutions.

Table 17: CAST k:n structured-projection results for parameter-to-compute conversion. Entries are post-FT ImageNet-1k top-1 under the source checkpoints and sparse-execution MAC assumptions described in the surrounding text. "Source s" is the unstructured Hybrid Magnitude–SER source sparsity; "dense" means the projection starts from the dense checkpoint. "MAC red." is the dense-equivalent sparse-execution MAC reduction, not raw parameter sparsity. "Th. spd." is the theoretical value $1/(1-\text{MAC red.})$. "Deploy spd." is a measured sparse-backend speedup for the audited checkpoint when available; Linear rows use the A40 native 2:4 endpoint at batch 128, and ResNet rows marked ‡ use exact im2col+2:4 sparse-GEMM relative to dense im2col. A dash means no exact sparse-backend wall-clock speedup is claimed for that row. Δ is top-1 minus the corresponding dense baseline, in percentage points.

Architecture	Source s	Method	Dense MACs	MAC red.	Th. spd.	Deploy spd.	Top-1 (%)	Δ (pp)
ViT family (CAST = cert-aware 2:4 + free restoration + ToMe $r = 8$ + 3-ep distill FT)								
ViT-B/16	0.35	Magnitude 2:4 + ToMe	17.56G	59.81%	2.49×	1.36×*	82.92	−2.19
ViT-B/16	0.35	**CAST**	17.56G	59.81%	2.49×	1.36×	**83.41**	−1.70
ViT-B/16	0.35	**CAST 6:12 SER+α=0.5** (no ToMe)	17.56G	50%	2.00×	–	**83.74**§	−1.37
ViT-L/16	0.35	CAST	61.55G†	∼60%†	2.50×	1.37×	**84.37**	−1.47
ViT-L/16	dense	**CAST 8:16 dense+perm** (no ToMe)	61.55G	50%	2.00×	–	**85.33**§	−0.51
DeiT-B	0.35	CAST	17.56G	59.81%	2.49×	1.36×	80.48	−1.32
DeiT-S	0.35	CAST	4.61G	59.81%	2.49×	1.38×	76.96	−2.89
DeiT-T	0.35	CAST	1.26G	59.81%	2.49×	1.33×	65.93	−6.28
ResNet family (CAST-conv = cert-aware 1×1+3×3 2:4 + free restoration; ToMe N/A)								
ResNet50	0.35	CAST-conv	4.09G	48.5%	1.94×	–	73.14	−2.99
ResNet50	0.35	**CAST-conv+perm**	4.09G	48.5%	1.94×	1.70×‡	**75.67**‡	−0.46
ResNet50	dense	**CAST 8:16 dense+perm**	4.09G	50%	2.00×	–	**75.87**§	−0.26
ResNet50d	0.35	CAST-conv	4.33G	49.85%	1.99×	–	78.08	−2.47
ResNet50d	0.35	**CAST-conv+perm**	4.33G	49.85%	1.99×	1.50×‡	**78.00**‡	−2.55
ResNet50d	dense	**CAST 8:16 dense+perm**	4.33G	50%	2.00×	–	**78.57**§	−1.98
ResNet101d	0.35	CAST-conv	8.0G	∼50%	2.00×	–	80.13	−2.13
ResNet101d	0.35	**CAST-conv+perm**	8.0G	∼50%	2.00×	1.57×‡	**80.59**‡	−1.67
ResNet101d	dense	**CAST 8:16 dense+perm**	8.0G	50%	2.00×	–	**80.92**§	−1.34
ResNet152d	0.35	**CAST-conv+perm**	11.8G	∼50%	2.00×	1.62×‡	**81.33**‡	−1.53
ConvNeXt family (CAST on the nn.Linear pointwise convs; ToMe N/A)								
ConvNeXtV2-Base	0.35	**CAST 2:4 cert + free-restore**	15.4G	50%	2.00×	1.26×	**85.47**	−1.25
ConvNeXtV2-Base	dense	**CAST 12:16 dense+perm** (25% sparse)	15.4G	25%	1.33×	–	**86.35**§	−0.37
ConvNeXtV2-Base	dense	**CAST 8:16 dense+perm** (50% sparse)	15.4G	50%	2.00×	–	**85.85**§	−0.87

†ViT-L/16 reduction is estimated from the analytical 22.81% ToMe reduction plus the 50% 2:4 Linear share.
‡**CAST-conv+perm** uses the importance-balanced Cin permutation alignment of Section 3.2; deploy speedups on these rows are exact im2col+2:4 sparse-GEMM versus dense im2col, while sparse im2col remains slower than cuDNN Conv2d end-to-end. §Rows marked § are wider-pattern or lower-sparsity accuracy probes; their speedups are dense-equivalent arithmetic estimates unless a native kernel is measured. *The magnitude row uses the same ViT-B 2:4+ToMe deployment path as the CAST row; the deployability audit located the pre-FT magnitude 2:4 artifact but not a separate final post-FT magnitude checkpoint.

Table 18: Mapping from CAST/CAST-conv+perm pipeline steps to the certificate theorems of Section **??**. Each row pairs an implementation component (left) with the theorem, lemma, or corollary that motivates it (right). The table records theorem-side motivations and the empirical proxy each step is intended to reduce; it does not state that the implemented pipeline verifies the literal certificate inequalities.

Pipeline step	Theorem / lemma in paper
SER $s = 0.35$ starting checkpoint	Marchenko–Pastur edge σ_+ as bulk/spike threshold (Cor. **??**); entries below σ_+ are expected to have small certificate cost under the Gaussian sufficient condition.
Per-layer pruning budget (SEB allocator)	Cor. **??** — high-σ_+ layers are the random-capacity reservoirs; allocate more pruning there because $B_T(R_\ell)$ responds most weakly per unit removed Frobenius mass.
Cert-aware 2:4 mask (CAST core)	Lemma **??** — pick the 2-of-4 keep pattern that minimises the activation-weighted removed-contribution cost $B_T(R)$ per output row, rather than sharing one pattern across rows.
Free restoration	Theorem **??** and Corollary **??** — splitting the removed mass into a kept reservoir Q and a finally-removed component P; restoration is certificate-improving when the reduction $B_T(R) - B_T(P)$ exceeds $\lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. The implementation uses this as motivation for zero-FLOP restoration inside structured groups.
α_{ser} Hamming prior	Theorem **??** + an empirical Bayesian prior on the certificate-improving Q: bias the 2:4 keep pattern toward the SER-validated unstructured mask, since SER kept entries were selected by an MP-edge-guided pruning process.
Permutation alignment (CAST-conv+perm)	Cin importance balancing minimises the dispersion of B_T contributions across 4-tuples; the importance-balanced quartile interleave of Section 3.2 ("flatten the layer") is designed to reduce the max-per-group certificate cost.

Table 19: ViT-B/16 reference at $\sim 30\%$ compression on ImageNet-1k validation. CP-ViT values are approximate because they are read from the published figure; the comparison is indicative rather than apples-to-apples because of the checkpoint mismatch (CP-ViT baseline 77.91% vs. our 85.11%). Hybrid Magnitude–SER and the three within-paper baselines are reproduced from Table **??**. Bold method rows mark this paper's rows only, not better performance. All entries are post-fine-tuning unless noted; CP-ViT used 30 FT epochs vs. our cumulative ~ 6 FT epochs through $s = 0.30$.

Method	Top-1 (%)	Δ (pp)	Compression
ViT-B/16 [11] (our dense baseline 85.11; CP-ViT baseline 77.91)			
CP-ViT [29], no FT*	~ 76.4	~ -1.5	$\sim 30\%$ FLOPs
CP-ViT [29], with FT*	~ 77.0	~ -0.9	$\sim 30\%$ FLOPs
Classical magnitude (this paper)	84.46	−0.65	30% param
Classical RMT (this paper)	84.55	−0.56	30% param
SER (this paper)	84.55	−0.56	30% param
Hybrid Magnitude–SER (this paper)	**84.64**	**−0.47**	**30%** param
ViT-B/16 at higher compression (only this paper has direct numbers)			
Classical magnitude, $s = 0.50$	81.67	−3.44	50% param
Classical RMT, $s = 0.50$	82.57	−2.54	50% param
SER, $s = 0.50$	83.27	−1.84	50% param
Hybrid Magnitude–SER, $s = 0.50$	**83.37**	**−1.74**	**50%** param
ViT-B/16 at high compression (only this paper has direct numbers)			
Classical magnitude, $s = 0.65$	74.30	−10.81	65% param
Classical RMT, $s = 0.65$	76.46	−8.65	65% param
SER, $s = 0.65$	73.80	−11.31	65% param
Hybrid Magnitude–SER, $s = 0.65$	**79.95**	**−5.16**	**65%** param
Hybrid Magnitude–SER, $s = 0.70$	**78.01**	**−7.10**	**70%** param
Classical magnitude, $s = 0.70$	67.44	−17.67	70% param
Classical RMT, $s = 0.70$	71.53	−13.58	70% param

*Approximate values read from the CP-ViT figure; CP-ViT used a different (77.91%) ViT-B/16 checkpoint. Numerical entries for our methods are from Table **??**.

Table 20: DeiT pruning comparison after fine-tuning. This is not an apples-to-apples ranking: rows use different dense baselines, compression axes, and training protocols. "Top-1" is the post-pruning ImageNet-1k validation accuracy and "Δ" is the absolute change from the dense baseline used by that method. "Compression" is FLOPs savings for VTP/PoWER/HVT/CP-ViT and parameter sparsity for Hybrid Magnitude–SER (see protocol caveats above). VTP, PoWER, and HVT DeiT numbers are reproduced from CP-ViT [29]. Bold method rows mark this paper's rows only, not better performance. Hybrid Magnitude–SER entries are included only where an archived full-validation checkpoint is available.

Method	Top-1 (%)	Δ (pp)	Compression
DeiT-Tiny 16/224 [32] (prior dense 72.20; our dense 72.21)			
VTP [40]†	70.55	−1.65	45.32% FLOPs
PoWER-BERT [14]†	70.05	−2.15	41.26% FLOPs
HVT [25]†	70.01	−2.19	47.32% FLOPs
CP-ViT [29]	71.24	−0.96	43.34% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**71.86**	**−0.35**	**20%** param
DeiT-Small 16/224 [32] (prior dense 79.80; our dense 79.85)			
VTP [40]†	78.24	−1.56	42.52% FLOPs
PoWER-BERT [14]†	78.30	−1.50	41.36% FLOPs
HVT [25]†	78.05	−1.75	47.80% FLOPs
CP-ViT [29]	79.08	−0.72	42.24% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**79.37**	**−0.48**	**20%** param
Hybrid Magnitude–SER, $s = 0.40$ **(this paper)**	78.61	−1.24	40% param
DeiT-Base 16/224 [32] (prior dense 81.82; our dense 81.80)			
VTP [40]†	80.70	−1.12	43.20% FLOPs
PoWER-BERT [14]†	80.17	−1.65	39.24% FLOPs
HVT [25]†	79.94	−1.88	44.78% FLOPs
CP-ViT [29]	81.13	−0.69	41.62% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**81.46**	**−0.34**	**20%** param
Hybrid Magnitude–SER, $s = 0.35$ **(this paper)**	81.17	−0.63	35% param
Hybrid Magnitude–SER, $s = 0.40$ **(this paper)**	80.99	−0.81	40% param

†Reproduced by CP-ViT [29]; not reported in the original VTP/PoWER/HVT papers.